سُبْحَانَكَ لاَ عِلْمَ لَنَا إِلاَّ مَا عَلَّمْتَنَا ۖ إِنَّكَ أَنْتَ الْعَلِيمُ الْحَكِيمُ

Dedications

In the name of Allah, the Most Gracious, the Most Merciful

To our beloved parents,

whose unwavering support, endless sacrifices, and constant prayers
have been the foundation of our success.

To our teachers and mentors,
who guided us with knowledge and wisdom
throughout our academic journey.

To the team at Novation City,
who welcomed us with open arms and provided
invaluable learning opportunities.

To our families and friends,
whose encouragement and understanding
made this achievement possible.

With deep gratitude and appreciation.

Acknowledgments

I would like to express my sincere gratitude to all those who contributed to the success of this professional internship experience, including academic supervisors, industry mentors, and institutional staff whose guidance, feedback, and practical support were essential to achieving the project objectives.

First, I extend my deepest appreciation to my academic supervisor, **Mr. Ahmed Bin Ayed**, for their invaluable guidance and continuous support throughout this project. Their expertise was instrumental in ensuring academic rigor and alignment with university requirements.

I am profoundly grateful to the Novation City team who welcomed me into their innovative ecosystem. Special thanks to my professional supervisor, **Mr. Mohamed Chrifa**, for their professional insights and technical guidance in Industry 4.0 implementation and digital transformation consulting.

My sincere appreciation goes to Novation City competitiveness cluster for entrusting me with developing the **IoT-REAP (Internet of Things - Remote Equipment Access Platform)**. Building this secure industrial remote operations platform—integrating Proxmox VE virtualization, Apache Guacamole remote desktop, MQTT-based IoT robot control, and AI-powered demand forecasting—has been an invaluable experience for my professional growth in full-stack development and industrial systems integration.

I extend gratitude to EPI Digital School's administration and faculty for providing quality education and academic resources essential for understanding enterprise-grade web application development, security architecture, and modern software engineering practices.

Finally, I am grateful to my family and friends for their unwavering support and encouragement throughout this journey.

Contents

Dedications . . . . . **i**

Acknowledgments . . . . . **ii**

List of Figures . . . . . **v**

List of Tables . . . . . **vi**

Chapter 1: Conceptual Analysis **1**

Introduction 2

1 Identification of the Actors 2

2 Requirements Specifications 3

2.1 Functional Requirements 3

3 Use Case Diagram 6

3.1 General Use Case Diagram 6

3.2 Refinement and Specification of Use Cases 9

3.2.1 Guest Use Cases 9

3.2.1.1 Browse Training Content 9

3.2.1.2 Authenticate 12

3.2.1.3 Verify a certificate 18

3.2.2 Engineer Use Cases 20

3.2.2.1 Follow Training Paths 20

3.2.2.2 Manage Reservations 27

3.2.2.3 Manage Virtual Lab 30

3.2.2.4 Handle Payments 36

3.2.2.5 Manage Account 38

3.2.3 Teacher Use Cases 41

3.2.3.1 Manage Training Paths 42

3.2.3.2 Manage Forum Interactions 47

3.2.3.3 Manage VM Assignments 50

3.2.4 Administrator Use Cases 52

3.2.4.1 Consult Dashboard & Platform Analytics 53

3.2.4.2 Manage Users & Roles 57

3.2.4.3 Manage Infrastructure and Hardware 61

3.2.4.4 Set Maintenance Mode 71

3.2.4.5 Manage Training Paths 75

3.2.4.6 Manage Reservations 80
3.2.4.7 Manage Payout & Refunds 84
4 Sequence Diagrams 88
4.1 Training Path Enrollment and Checkout Sequence 88
4.2 Teacher Submission for Review Sequence 89
4.3 Virtual Machine Provisioning Sequence 89
4.4 Hardware Attachment and Queue Sequence 90
4.5 Quiz Attempt Submission Sequence 92
4.6 Google OAuth Authentication Sequence 93
4.7 MQTT Robot Control Sequence 94
4.8 Camera PTZ Control Sequence 94
4.9 Refund Processing Sequence 96
5 Class Diagram 97
5.1 User and Access Domain 97
5.2 Learning Content Domain 98
5.3 Commerce and Revenue Domain 99
5.4 Virtual Laboratory and Infrastructure Domain 99
6 Physical Data Model 100
6.1 Learning and Authoring Tables 100
6.2 Participation and Certification Tables 101
6.3 Payment and Revenue Tables 101
6.4 Virtual Machine and Infrastructure Tables 101
6.5 Hardware and Reservation Tables 101
6.6 Governance and Monitoring Tables 102
7 Component Architecture Diagram 102
8 Deployment Diagram 103
Conclusion 104

List of Figures

1.1 General Use Case Diagram 8
1.2 Use Case Browse Training Content 9
1.3 Use Case Authenticate 13
1.4 Use Case Verify a certificate 19
1.5 Use Case Follow Training Paths 21
1.6 Use Case Manage Reservations 27
1.7 Use Case Manage Virtual Lab 30
1.8 Use Case Handle Payments 36
1.9 Use Case Manage Account 39
1.10 Use Case Manage Training Paths 42
1.11 Use Case Manage Forum Interactions 47
1.12 Use Case Consult Dashboard & Platform Analytics 53
1.13 Use Case Manage Users & Roles 57
1.14 Use Case Manage Infrastructure and Hardware 62
1.15 Use Case Set Maintenance Mode 72
1.16 Use Case Manage Training Paths 76
1.17 Use Case Manage Reservations 80
1.18 Use Case Manage Payout & Refunds 84
1.19 Sequence Diagram: Training Path Enrollment and Checkout 88
1.20 Sequence Diagram: Teacher Submission for Review 89
1.21 Sequence Diagram: Virtual Machine Provisioning 90
1.22 Sequence Diagram: Hardware Attachment and Queue 91
1.23 Sequence Diagram: Quiz Attempt Submission 92
1.24 Sequence Diagram: Google OAuth Authentication 93
1.25 Sequence Diagram: MQTT Robot Control 94
1.26 Sequence Diagram: Camera PTZ Control 95
1.27 Sequence Diagram: Refund Processing 97
1.28 Class Diagram of the IoT-REAP Platform 100
1.29 Entity-Relationship Diagram of the IoT-REAP Physical Data Model 102
1.30 Component Architecture Diagram of the IoT-REAP Platform 103
1.31 Deployment Diagram of the IoT-REAP Platform 104

List of Tables

1.1 Use Case Specification: Browse and Discover Training Paths 10
1.2 Use Case Specification: Read Legal Documents 11
1.3 Use Case Specification: Browse Public Forum 12
1.4 Use Case Specification: Authenticate with Google OAuth 14
1.5 Use Case Specification: Authenticate with Email & Password 15
1.6 Use Case Specification: Register Account (email or Google) 16
1.7 Use Case Specification: Reset Password via Email Link 17
1.8 Use Case Specification: Verify Email 18
1.9 Use Case Specification: Verify Certificate 19
1.10 Use Case Specification: Download Certificate by Hash Link 20
1.11 Use Case Specification: Manage Training Path Enrollment 22
1.12 Use Case Specification: Consume Training Unit Content 23
1.13 Use Case Specification: Participate in Forum 24
1.14 Use Case Specification: Manage Personal Notes 25
1.15 Use Case Specification: Assessment and Path Completion 26
1.16 Use Case Specification: Browse Reservations 28
1.17 Use Case Specification: Manage Reservation Lifecycle 29
1.18 Use Case Specification: Manage VM Sessions 31
1.19 Use Case Specification: Manage Session Devices 33
1.20 Use Case Specification: Access Laboratory Cameras 34
1.21 Use Case Specification: Control PTZ Camera 35
1.22 Use Case Specification: Purchase Course / Checkout 37
1.23 Use Case Specification: View Payment History and Request Refund 38
1.24 Use Case Specification: Edit Profile 39
1.25 Use Case Specification: Change Password 40
1.26 Use Case Specification: View & Manage Notifications 41
1.27 Use Case Specification: View Analytics & Performance 42
1.28 Use Case Specification: View Training Paths Dashboard 43
1.29 Use Case Specification: Manage Training Path Lifecycle 44
1.30 Use Case Specification: Manage Modules and Training Units 45
1.31 Use Case Specification: Manage Multimedia Learning Content 46
1.32 Use Case Specification: Browse Forum Inbox 48
1.33 Use Case Specification: Moderate Forum Threads 49
1.34 Use Case Specification: Submit VM Assignment 50
1.35 Use Case Specification: Manage VM Assignments 51

1.36 Use Case Specification: Manage Earnings & Payouts 52
1.37 Use Case Specification: View Admin Dashboard and Platform KPIs 54
1.38 Use Case Specification: Monitor Platform Health and Alerts 55
1.39 Use Case Specification: View and Filter Activity Logs 56
1.40 Use Case Specification: Browse and Inspect Users 58
1.41 Use Case Specification: Manage Account Status and Role 59
1.42 Use Case Specification: Manage Teacher Approval 60
1.43 Use Case Specification: Impersonate User and GDPR Data Deletion 61
1.44 Use Case Specification: Manage Proxmox Servers 63
1.45 Use Case Specification: Browse Infrastructure: Nodes and VMs 64
1.46 Use Case Specification: Control VM Power State 65
1.47 Use Case Specification: Manage Gateway Nodes 66
1.48 Use Case Specification: Manage Hardware Devices 68
1.49 Use Case Specification: Manage Cameras 70
1.50 Use Case Specification: Manage Connection Preferences 71
1.51 Use Case Specification: Manage USB Device Maintenance 73
1.52 Use Case Specification: Manage Camera Maintenance 74
1.53 Use Case Specification: Moderate Forum Threads 75
1.54 Use Case Specification: Review and Approve Training Paths 77
1.55 Use Case Specification: Curate Featured Training Paths 78
1.56 Use Case Specification: Manage VM Assignments 79
1.57 Use Case Specification: Manage USB Device Reservations 81
1.58 Use Case Specification: Manage Camera Reservations 83
1.59 Use Case Specification: Manage Payout Requests 85
1.60 Use Case Specification: Manage Refund Requests 87

Chapter 1

Conceptual Analysis

Introduction

This chapter presents the conceptual analysis of the current IoT REAP platform.Web-based learning and remote laboratory environment built around training paths, virtual machine sessions, hardware reservations, multimedia learning content, discussion features, and administrative infrastructure management. The chapter models the system through actor identification, requirements specification, use case analysis, sequence and activity modeling, class-level conceptualization, and the physical data model that supports platform execution.

1 Identification of the Actors

The platform is organized around role-based access control implemented in the application through the user roles `engineer`, `teacher`, and `admin`, in addition to the public unauthenticated state. Each actor interacts with the same platform from a different operational perspective, and each role is associated with a distinct set of services, permissions, and workflows.

Unauthenticated User (Guest) represents any visitor who has not yet authenticated in the system. This actor can browse public training paths, access legal and informational pages, consult public discussion threads in read-only mode, view training path reviews, verify public certificates, initiate authentication and registration workflows, and access public checkout result pages after external payment redirection. The guest actor therefore serves as the entry point into the platform ecosystem.

Engineer is the primary learner and remote laboratory user. This actor enrolls in training paths, consumes educational resources such as articles and videos, takes quizzes, tracks progress, writes notes, participates in discussions, posts reviews, claims certificates, and initiates payment and refund operations when a training path is monetized. Beyond the learning dimension, the engineer also provisions virtual machine sessions, opens remote desktop access through Guacamole, configures connection preferences, reserves USB devices and cameras, attaches hardware to active sessions, enters waiting queues when devices are busy, and interacts with session-bound lab resources.

Teacher is the content producer and pedagogical supervisor. This actor creates and manages training paths, modules, and training units; authors articles; uploads and manages videos and captions; creates and publishes quizzes; reviews learner interactions through analytics dashboards; moderates instructional discussions; submits training paths for administrative approval; and requests payout for generated revenue. The teacher role links educational design with operational control over learning content.

Administrator is the platform governance and infrastructure actor. This role supervises users, teacher approval, training path approval, featured training path ordering, payment-related review processes, payout validation, infrastructure supervision, Proxmox server and node management, VM assignment approval, hardware gateway verification, camera and reservation management, moderation of flagged forum content, maintenance operations, alerts, and activity logging. Administrators also have access to impersonation and compliance-sensitive actions such as suspension and GDPR-oriented user deletion.

Stripe (External Payment Processor) is an external system actor that supports the platform commerce workflows. IoT-REAP integrates with Stripe through hosted checkout redirections and signed webhook callbacks to confirm payments, create internal payment records, and react to asynchronous state changes. Stripe also participates in finance workflows such as refunds and instructor payouts, where administrators trigger payout transfers and refund executions through Stripe APIs while preserving auditability and transaction integrity.

The interaction among these actors reveals that the platform combines four complementary dimensions: e-learning delivery, instructor authoring, virtual laboratory provisioning, and infrastructure governance. This combination distinguishes the project from a conventional learning management system because educational workflows are directly connected to remote computing and physical IoT resource access.

2 Requirements Specifications

This section formalizes the system requirements derived from the implemented routes, services, and domain models. The requirements are divided into functional and non-functional categories in order to describe both what the platform must do and how well it must perform those responsibilities.

2.1 Functional Requirements

The platform must support the following functional requirements according to actor responsibilities and shared system behavior.

Unauthenticated User (Guest):

- Access account registration and login workflows
- Consult the landing page and public legal pages
- Browse public training paths and training path details
- Read public reviews and forum threads in read-only mode

- Verify and download publicly accessible certificates through verification links
- View public search results, trending queries, and category-based discovery
- Access checkout success and cancellation pages after external payment flows

Engineer:

- Enroll in and unenroll from training paths
- Access owned or enrolled training units and learning materials
- Read articles and track article completion
- Stream instructional videos and update playback progress
- Attempt quizzes, submit answers, and review attempt history
- Mark training units complete or incomplete based on learning progress
- Create, update, and delete personal notes for training units
- Create, edit, and delete reviews for completed or relevant training paths
- Participate in forum discussions by creating threads, replying, voting, and flagging content
- View notifications and manage read status
- Claim, verify, and download certificates
- Initiate checkout for paid training paths
- Review payment history and submit refund requests
- Provision, extend, view, and terminate virtual machine sessions
- Retrieve Guacamole access tokens for remote desktop connectivity
- Manage connection preference profiles for available protocols
- Browse available Proxmox virtual machines and snapshots
- Reserve USB devices and cameras
- Attach or detach session hardware and join hardware waiting queues
- Access session camera streams, request control, move PTZ-enabled cameras, and change stream resolution

Teacher:

- Create, edit, archive, restore, and delete training paths
- Organize training paths into modules and training units
- Reorder modules and training units
- Create and maintain quiz structures, questions, and publication status
- Create, update, and delete article content
- Upload, retry, and delete training videos
- Manage video captions
- Submit training paths for administrative review
- View teacher analytics for enrollment, earnings, and learning funnel data
- Access a teacher discussion inbox and moderate course-related threads
- Request revenue payout
- Submit training unit virtual machine assignment requests

Administrator:

- Access the administrative dashboard and KPI views
- Approve, reject, feature, unfeature, and order training paths
- Review pending VM assignment requests
- Manage reservations for devices and cameras
- Block or lock a USB device or camera to prevent new reservations (maintenance or administrative hold), or toggle an "open-for-all" mode to allow immediate non-reserved access
- Manage Proxmox servers and nodes
- Start, stop, reboot, and shut down virtual machines through node operations
- Register, verify, update, and remove gateway nodes
- Discover hardware devices and refresh infrastructure status
- Bind, unbind, attach, detach, dedicate, and configure devices and cameras

- Assign and unassign cameras to virtual machines
- Approve teacher accounts and manage user roles
- Suspend, unsuspend, impersonate, and delete user accounts where authorized
- Review and process refund requests
- Review, approve, reject, and process payout requests
- Monitor video processing status
- Place devices and cameras in maintenance mode
- Moderate flagged threads and replies
- Manage system alerts and consult activity logs

3 Use Case Diagram

After identifying actors and requirements, the conceptual analysis of the platform can be refined through use case modeling. The implemented application suggests one general use case view and several actor-specific refinements. The general model places the platform at the center and connects the four user actors (Guest, Engineer, Teacher, Administrator) as well as the external Stripe actor to the services they invoke: public discovery, learning progression, content authoring, infrastructure access, commerce, and administrative governance.

3.1 General Use Case Diagram

The general use case diagram groups system behavior into four principal subsystems:

- **Public Access and Discovery:** registration, authentication, public browsing, search, certificate verification
- **Learning and Participation:** enrollment, content consumption, quizzes, notes, reviews, forum participation, notifications, certificates
- **Teaching and Publishing:** training path authoring, multimedia management, quiz management, analytics, payout requests, submission for review
- **Remote Laboratory and Administration:** VM provisioning, remote access, reservation management, device control, infrastructure supervision, moderation, refunds, payouts, alerts, and logs

In this model, the Engineer extends the Guest by inheriting public consultation capabilities and adding authenticated learning and virtual laboratory operations. The Teacher extends the Engineer's authenticated access with content authoring and instructional management. The Administrator occupies the supervisory position by controlling both business governance and infrastructure operations. Stripe is modeled as a standalone external actor that interacts with the platform through checkout, webhook callbacks, refunds, and payout-related flows. Figure 1.1 presents the general use case diagram of the IoT-REAP platform, showing the four user actors plus Stripe and their principal interactions with the system subsystems.

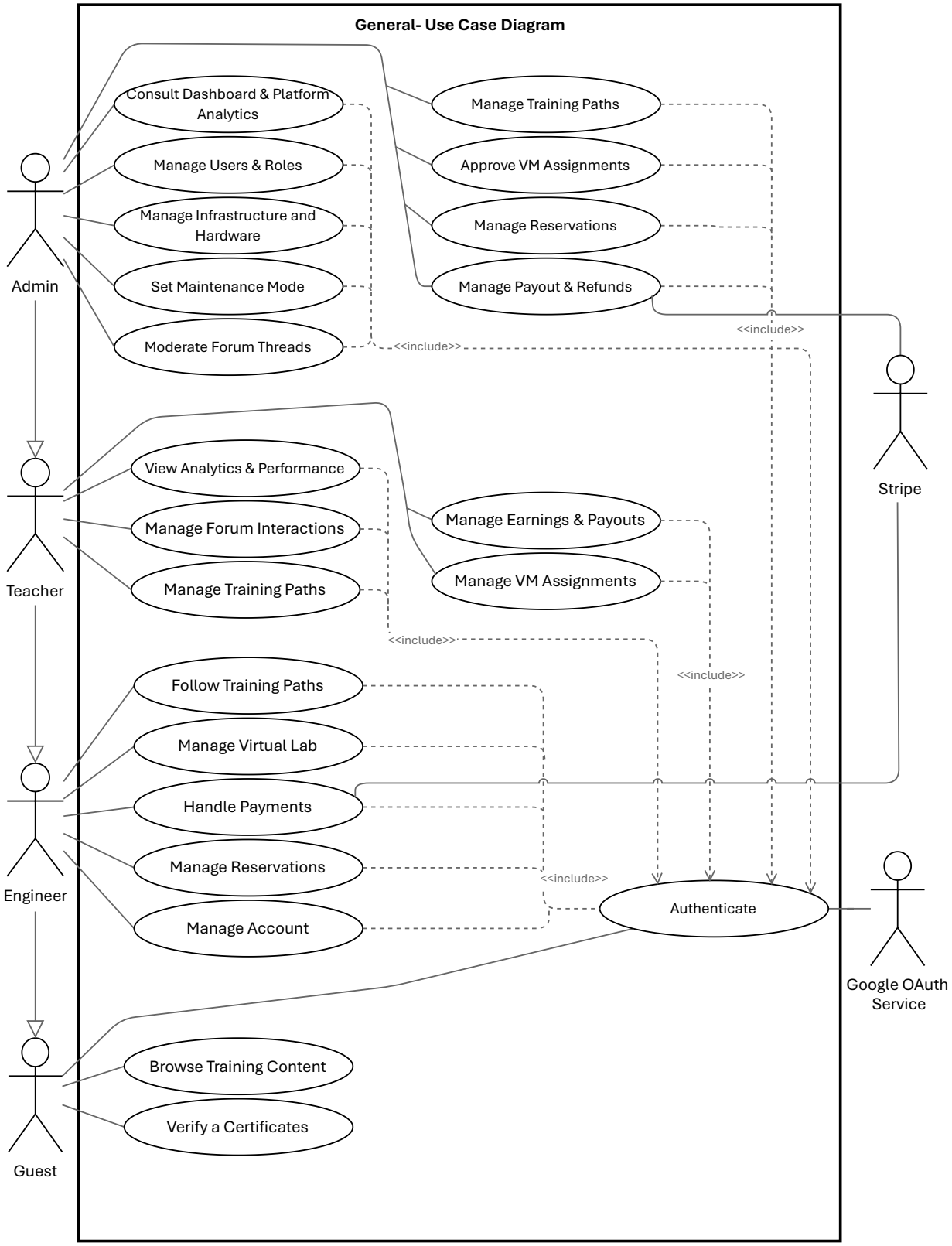

Figure 1.1: General Use Case Diagram

3.2 Refinement and Specification of Use Cases

The major use cases can be refined through representative specifications aligned with the actual implemented routes and domain behaviors.

3.2.1 Guest Use Cases

The Guest actor is restricted to public discovery and account access. The most important public use cases are account registration, training path consultation, certificate verification, and search.

3.2.1.1 Refinement of the Use Case Browse Training Content

Figure 1.2 presents the refinement of the Use Case Browse Training Content for the Guest actor. This use case includes the following sub-use cases:

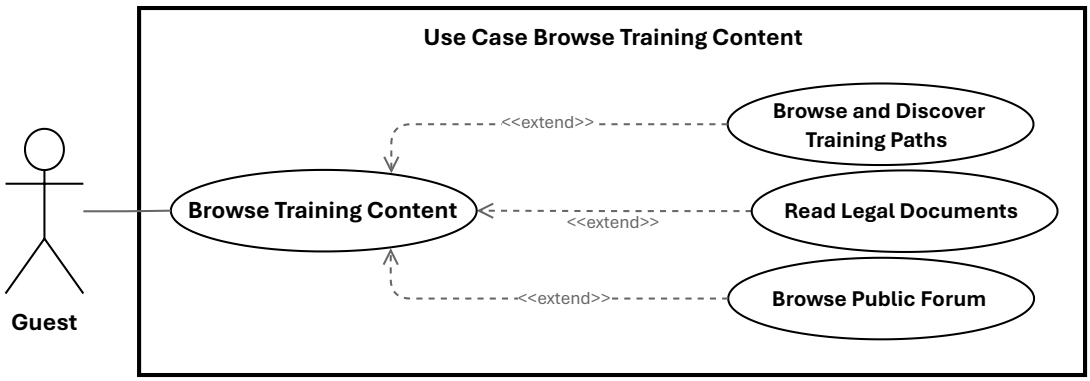

Figure 1.2: Use Case Browse Training Content

• **Specification of the Use Case Browse and Discover Training Paths**

Table 1.1: Use Case Specification: Browse and Discover Training Paths

Title	Browse and Discover Training Paths
Summary	Guest discovers available training paths by viewing the landing page, searching and filtering the catalog, and reading detailed path information including syllabus, reviews, pricing, and instructor details
Actor	Unauthenticated User (Guest)
Precondition	The platform is publicly accessible and at least one training path is published
Post-condition	Guest has viewed platform offerings and has enough information to decide whether to register
Basic Scenario	1. Guest arrives at the platform homepage; system displays featured training paths with titles, descriptions, and ratings 2. Guest navigates to the catalog; system lists all published training paths with categories, ratings, and pricing 3. Guest enters a keyword or selects category filters; system returns ranked matching results that may be sorted by price, rating, or enrollment count 4. Guest selects a training path; system displays the full detail page including description, learning objectives, module syllabus, pricing, duration, instructor profile, and enrolled-engineer statistics 5. Guest may read course reviews on the detail page to evaluate quality before deciding to register 6. If the guest attempts to enroll, the system redirects to the registration page
Error Scenario	1. If no featured paths are configured, the landing page displays a default welcome message 2. If no paths match the search criteria, the system displays a no-results message with suggestions 3. If the search service is unavailable, the system falls back to a static listing with a retry option 4. If a selected training path is not found or unpublished, the system redirects to the catalog

• **Specification of the Use Case Read Legal Documents**

Table 1.2: Use Case Specification: Read Legal Documents

Title	Read Legal Documents
Summary	Any user reads the platform's Terms of Service or Privacy Policy to understand usage rights and data handling practices
Actor	Unauthenticated User (Guest), Engineer, Teacher, Administrator
Precondition	The requested legal document is available and accessible
Post-condition	User views the complete and latest version of the selected legal document
Basic Scenario	1. User navigates to the legal section via footer links 2. User selects either **Terms of Service** or **Privacy Policy** 3. System retrieves the latest version of the selected document 4. System renders the document in a readable, properly formatted view 5. User scrolls through and reads the complete document
Error Scenario	1. If the document is unavailable or fails to load, the system displays a friendly error message with a retry option

• **Specification of the Use Case Browse Public Forum**

Table 1.3: Use Case Specification: Browse Public Forum

Title	Browse Public Forum
Summary	Guest browses and reads public forum threads and their replies to follow community discussions without authentication
Actor	Unauthenticated User (Guest)
Precondition	The platform is publicly accessible and the forum contains at least one published thread
Post-condition	Guest has read one or more forum threads and their approved replies
Basic Scenario	1. Guest navigates to the public forum page; system displays a paginated list of threads with titles, authors, and reply counts 2. Guest may sort threads by date, popularity, or category, or search by keyword 3. Guest selects a thread; system displays the full detail page showing the original post with author and timestamp, followed by all approved replies in chronological order 4. Guest reads the discussion and may view engagement metrics 5. If the guest attempts to post or reply, the system redirects to the registration page
Error Scenario	1. If no threads match a search query, the system displays a no-results message 2. If a selected thread is not found or has been removed, the system displays a not-found notice and returns to the listing

3.2.1.2 Refinement of the Use Case Authenticate

Figure 1.3 presents the refinement of the Use Case Authenticate for the Guest actor. This use case includes the following sub-use cases:

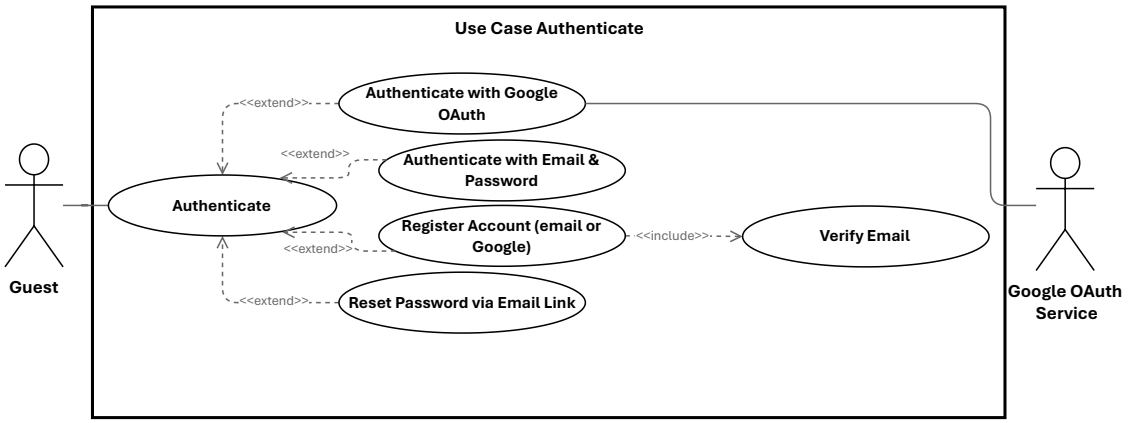

Figure 1.3: Use Case Authenticate

- **Specification of the Use Case Authenticate with Google OAuth**

Table 1.4: Use Case Specification: Authenticate with Google OAuth

Title	Authenticate with Google OAuth
Summary	Guest authenticates to the platform using a Google account through the OAuth 2.0 protocol, enabling single sign-on without manual credential entry
Primary Actor	Unauthenticated User (Guest)
Secondary Actor	Google OAuth Service
Precondition	The guest has a valid Google account; the Google OAuth integration is configured and active
Post-condition	The guest is authenticated and redirected to the role-appropriate dashboard, or prompted to select a role if a new account
Basic Scenario	1. Guest clicks the Google Sign-In button on the login or registration page 2. System redirects the guest to the Google OAuth consent screen 3. Guest authorizes the application and Google returns an authorization code 4. System exchanges the authorization code for user profile information 5. If the Google account matches an existing user, the system logs the user in 6. If the Google account is new, the system creates a pending account and presents the role selection page 7. Guest selects a role (Engineer or Teacher) and completes the signup 8. System authenticates the user and redirects to the appropriate dashboard
Error Scenario	1. If the Google OAuth callback fails, the system redirects to the login page with an error message 2. If the guest denies consent on the Google screen, the system returns to the login page 3. If the Google account email is already linked to a password-based account, the system prompts the user to log in with email and password

• **Specification of the Use Case Authenticate with Email & Password**

Table 1.5: Use Case Specification: Authenticate with Email & Password

Title	Authenticate with Email & Password
Summary	Guest authenticates to the platform using email and password to access protected features
Actor	Guest
Precondition	The guest has a registered and verified account
Post-condition	The guest is authenticated and receives a session with their assigned role
Basic Scenario	1. Guest navigates to the login page 2. System displays the login form with email and password fields and a Google OAuth button 3. Guest enters their credentials and submits the form 4. System validates the credentials against the stored hash 5. System creates an authenticated session with the user's role 6. System redirects the guest to their role-specific dashboard
Error Scenario	1. If the credentials are invalid, the system displays an error and increments the failed attempt counter 2. If the account is locked due to excessive failed attempts, the system prompts the guest to reset their password 3. If Google OAuth fails, the system falls back to the email/password login form

• **Specification of the Use Case Register Account (email or Google)**

Table 1.6: Use Case Specification: Register Account (email or Google)

Title	Register Account (email or Google)
Actor	Guest
Precondition	The guest is not authenticated and the platform allows new registrations
Post-condition	A new user account is created with pending email verification status
Basic Scenario	1. Guest navigates to the registration page 2. System displays the registration form with required fields 3. Guest enters their name, email address, and password 4. System validates the input fields and checks for duplicate email 5. System creates the account with pending verification status 6. System sends a verification email to the provided address 7. Guest clicks the verification link to activate the account
Error Scenario	1. If the email is already registered, the system prompts the guest to log in instead 2. If the password does not meet security requirements, the system rejects the submission 3. If the verification email expires, the system allows resending a new link

• **Specification of the Use Case Reset Password via Email Link**

Table 1.7: Use Case Specification: Reset Password via Email Link

Title	Reset Password via Email Link
Summary	Guest, engineer, or teacher resets their password via email link
Actor	Unauthenticated User (Guest), Engineer, Teacher
Precondition	The user has an active account with a registered email
Post-condition	The user's password is updated and they can log in with the new password
Basic Scenario	1. User clicks "Forgot Password" on the login page 2. User enters their email address 3. System validates the email and creates a password reset token 4. System sends a password reset email with a secure link 5. User clicks the link and is redirected to the reset form 6. User enters and confirms a new password 7. System validates the password meets security requirements 8. System updates the password and invalidates the reset token 9. System notifies the user via email that the password was changed
Error Scenario	1. If the email is not registered, the system displays a generic success message (security) 2. If the reset token expires, the system prompts the user to request a new one 3. If the new password does not meet requirements, validation fails

• **Specification of the Use Case Verify Email**

Table 1.8: Use Case Specification: Verify Email

Title	Verify Email
Summary	Guest, engineer, or teacher verifies their email address to activate their account
Actor	Unauthenticated User (Guest), Engineer, Teacher
Precondition	The user has registered an account but not verified their email
Post-condition	The user's email is marked as verified and full account access is granted
Basic Scenario	1. User registers a new account or logs in with an unverified email 2. System creates an email verification token 3. System sends a verification email with a secure link 4. User clicks the link and is redirected to the verification endpoint 5. System validates the token and marks the email as verified 6. System notifies the user that verification was successful 7. User is redirected to their dashboard with full access
Error Scenario	1. If the token expires, the system prompts the user to request a new one 2. If the token is invalid, the system displays an error message 3. If the email is already verified, the system notifies the user

3.2.1.3 Refinement of the Use Case Verify a certificate

Figure **??** presents the refinement of the Use Case Verify a certificate for the Guest actor. This use case includes the following sub-use cases:

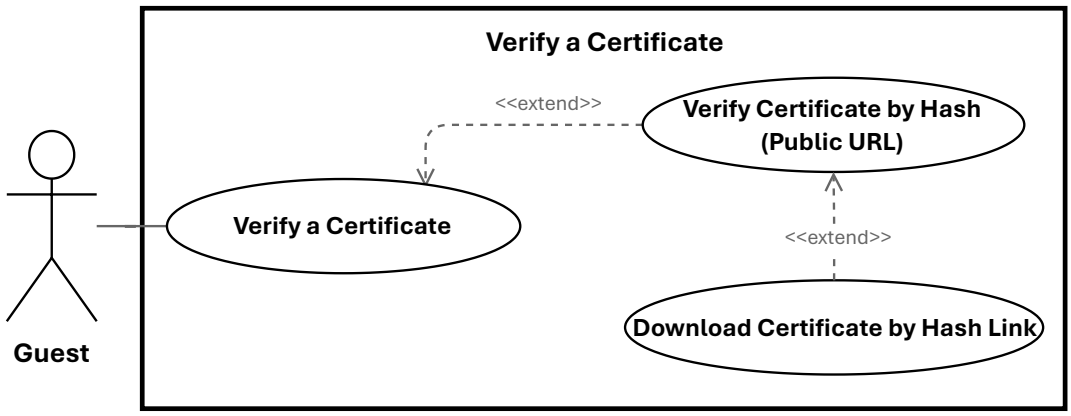

Figure 1.4: Use Case Verify a certificate

• **Specification of the Use Case Verify a Certificate**

Table 1.9: Use Case Specification: Verify Certificate

Title	Verify Certificate
Summary	Guest verifies the authenticity of a digital certificate using its hash
Actor	Unauthenticated User (Guest)
Precondition	The certificate exists and has been issued by the platform
Post-condition	The guest receives confirmation of the certificate's validity and details
Basic Scenario	1. Guest receives a certificate (e.g., via email or LinkedIn) 2. Guest navigates to the certificate verification page 3. Guest enters the certificate hash (or clicks a verification link) 4. System validates the hash and retrieves the certificate record 5. System displays the certificate details: recipient name, issue date, and credential title 6. System confirms the certificate is authentic and not revoked
Error Scenario	1. If the hash is invalid, the system displays an error message 2. If the certificate is revoked, the system notifies the guest 3. If the certificate has expired, the system displays an expiration warning

• **Download Certificate by Hash Link**

Table 1.10: Use Case Specification: Download Certificate by Hash Link

Title	Download Certificate by Hash Link
Summary	Guest downloads a digital certificate using a verification hash link without authentication
Actor	Unauthenticated User (Guest)
Precondition	The certificate exists, is valid, and has not been revoked
Post-condition	The guest downloads the certificate file in PDF or image format
Basic Scenario	1. Guest receives a certificate download link via email or external platform 2. Guest clicks the download link containing the certificate hash 3. System validates the hash and retrieves the certificate record 4. System confirms the certificate is valid and not revoked 5. System generates and serves the certificate file for download 6. Guest saves the certificate file to their device
Error Scenario	1. If the hash is invalid, the system displays an error message 2. If the certificate is revoked, the system notifies the guest 3. If the certificate has expired, the system displays an expiration warning 4. If the download fails, the system offers a retry option

3.2.2 Engineer Use Cases

The Engineer actor focuses on training path enrollment, virtual lab access, and hands-on learning. The key use cases include training path enrollment and management, content consumption (videos, articles, quizzes), virtual machine session management, hardware device and camera control, reservation management, forum participation, certificate claims, payment operations, and account management.

3.2.2.1 Refinement of the Use Case Follow Training Paths

Figure 1.5 presents the refinement of the Use Case Follow Training Paths for the Engineer actor. This use case includes the following sub-use cases:

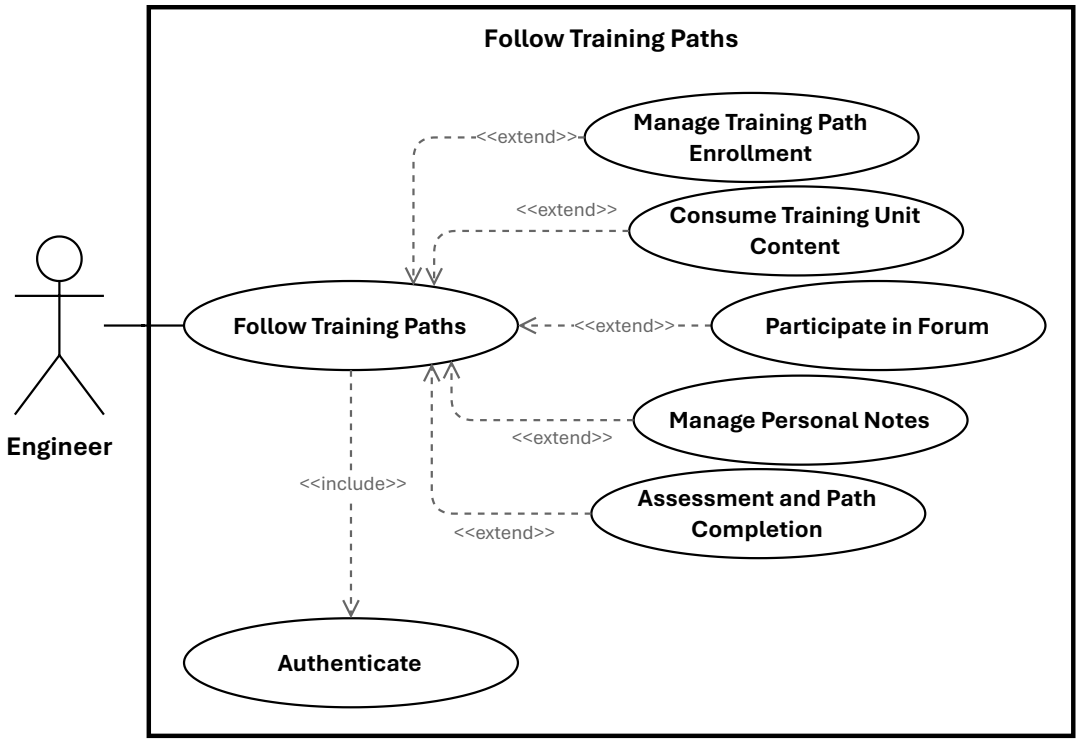

Figure 1.5: Use Case Follow Training Paths

- **Specification of the Use Case Manage Training Path Enrollment**

Table 1.11: Use Case Specification: Manage Training Path Enrollment

Title	Manage Training Path Enrollment
Summary	Engineer browses their enrolled training paths, enrolls in new ones, and unenrolls from paths they no longer wish to pursue
Actor	Engineer
Precondition	The engineer is authenticated and the target training path is published and available where applicable
Post-condition	The engineer's enrollment state is created or removed and access to training path content is granted or revoked accordingly
Basic Scenario	1. Engineer opens the dashboard; system displays all enrolled training paths with title, thumbnail, progress percentage, and status; engineer may filter by status or category 2. For **Enroll**: engineer selects a training path from the catalog and clicks Enroll; if the path is free enrollment is immediate; if paid the engineer is redirected to checkout; upon success the system creates the enrollment record and grants access to all modules and units 3. For **Unenroll**: engineer selects an enrolled training path and confirms the unenrollment action; system removes the enrollment record, revokes content access, and handles any applicable refund according to platform policy
Error Scenario	1. If the engineer has no enrollments, system displays an empty state with discovery suggestions 2. If the engineer is already enrolled, system displays a message and prevents duplicate enrollment 3. If the training path is unpublished or archived, enrollment is blocked and the path is marked accordingly 4. If payment fails, enrollment is not completed 5. If unenrollment is restricted due to completion status, system displays an explanatory message

• **Specification of the Use Case Consume Training Unit Content**

Table 1.12: Use Case Specification: Consume Training Unit Content

Title	Consume Training Unit Content
Summary	Engineer watches video lessons, reads article lessons, and marks training units complete or incomplete, with progress tracked and persisted throughout
Actor	Engineer
Precondition	The engineer is enrolled in the training path and the target training unit is accessible and published
Post-condition	The engineer's content progress is recorded and the overall training path progress is recalculated accordingly
Basic Scenario	1. Engineer opens a training unit from their enrolled training path 2. For **Video**: system loads the video player with the processed stream URL; engineer plays, pauses, seeks, or adjusts playback speed; system periodically records playback position; upon reaching the completion threshold the video is marked as watched 3. For **Article**: system renders the article with estimated reading time; engineer scrolls through the content; system tracks progress via scroll position or explicit confirmation; upon reaching the end the article is marked as read 4. Engineer may manually **mark the unit complete or incomplete** at any time; system persists the status and recalculates module and path-level progress percentages 5. Updated progress is reflected on the training path overview
Error Scenario	1. If the video has not finished processing, system displays a pending state with a retry option 2. If the video stream URL is unavailable, system shows an error with a fallback message 3. If the article is unpublished or fails to load, system shows a notice and prevents access 4. If progress saving fails, system retries silently and preserves the last known state 5. If the engineer is not enrolled, system denies access and progress updates

• **Specification of the Use Case Participate in Forum**

Table 1.13: Use Case Specification: Participate in Forum

Title	Participate in Forum
Summary	Engineer engages with training path discussion forums by posting threads or replies, upvoting helpful content, and flagging inappropriate content for moderation
Actor	Engineer
Precondition	The engineer is enrolled in at least one training path with an active forum
Post-condition	The engineer's post, upvote, or flag is persisted and reflected in the forum accordingly
Basic Scenario	1. Engineer navigates to the forum section of an enrolled training path 2. For **Post**: engineer creates a new discussion thread or replies to an existing post; system validates content, persists the post, and notifies subscribed participants 3. For **Upvote**: engineer clicks the upvote button on a thread or reply; system increments the upvote count and records the vote; engineer may remove their upvote at any time 4. For **Flag**: engineer clicks the flag button to report inappropriate content; system records the flag and may hide the content pending moderator review; engineer may remove their flag if desired
Error Scenario	1. If the forum is closed or locked, system prevents new posts and replies 2. If content validation fails, system displays specific error messages and preserves input 3. If the engineer has already upvoted, system prevents duplicate votes 4. If the content has already been flagged, system acknowledges the existing report

• **Specification of the Use Case Manage Personal Notes**

Table 1.14: Use Case Specification: Manage Personal Notes

Title	Manage Personal Notes
Summary	Engineer creates, edits, and deletes private notes attached to training units for personal reference
Actor	Engineer
Precondition	The engineer is enrolled in the training path and the training unit is accessible
Post-condition	The note is persisted, updated, or removed; notes remain private and are never visible to other users
Basic Scenario	1. Engineer opens a training unit; system displays any existing notes or an empty editor 2. Engineer writes or edits note content; system validates and persists the note on save 3. Engineer may delete a note at any time; system requests confirmation then removes it
Error Scenario	1. If note content exceeds the maximum length, validation fails with a specific message 2. If the engineer is not enrolled, system prevents note creation 3. If save or deletion fails, system preserves the current state and logs the error

• **Specification of the Use Case Assessment and Path Completion**

Table 1.15: Use Case Specification: Assessment and Path Completion

Title	Assessment and Path Completion
Summary	Engineer takes quiz assessments to test knowledge, writes a course review to share feedback, and claims a completion certificate upon finishing all training path requirements
Actor	Engineer
Precondition	The engineer is enrolled in the training path; the quiz is published for assessment; all modules and quizzes are completed before certificate claim
Post-condition	Quiz attempt is recorded and scored; review is persisted and visible after approval; certificate is generated and available on the engineer's profile
Basic Scenario	1. For **Quiz**: engineer opens a training unit containing a quiz; system displays instructions, question count, and time limit; engineer answers all questions and submits or time expires; system grades the attempt, displays results with correct answers and explanations, and updates training unit progress based on passing criteria 2. For **Review**: engineer navigates to the training path review section; selects a star rating and optionally writes a text review; system validates content, persists the review, recalculates the path average rating, and makes the review visible once approved; if a review already exists the system offers an edit option instead 3. For **Certificate**: engineer opens the completed training path overview; system verifies all completion requirements are met, generates a unique certificate with a verification hash, and makes it available for download, sharing, and display on the engineer's public profile
Error Scenario	1. If the quiz has already been attempted and retakes are not allowed, access is blocked 2. If quiz submission fails, system preserves answers and allows retry 3. If the rating is missing, system rejects the review submission 4. If review text violates content policies, system flags it for moderation 5. If completion requirements are not fully met, system displays the remaining items and blocks certificate generation 6. If certificate generation fails, system logs the error and allows retry

3.2.2.2 Refinement of the Use Case Manage Reservations

Figure 1.6 presents the refinement of the Use Case Manage Reservations for the Engineer actor. This use case includes the following sub-use cases:

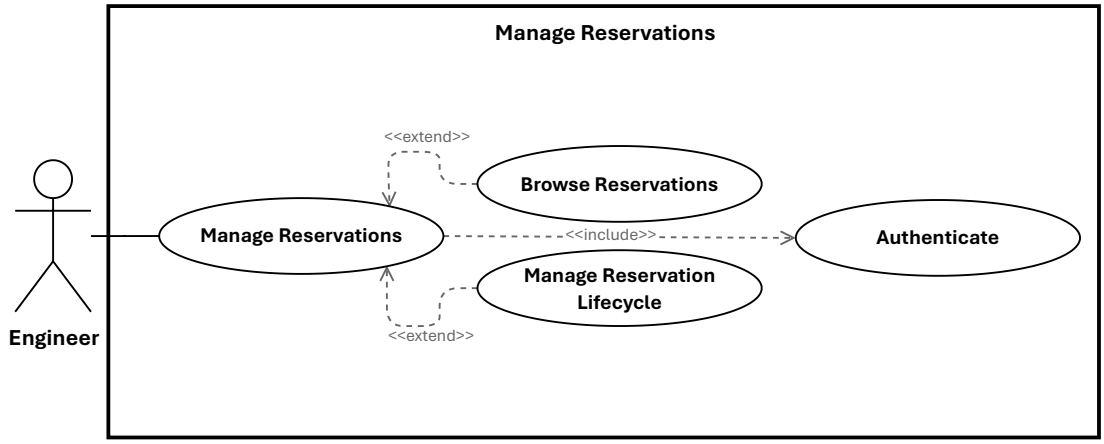

Figure 1.6: Use Case Manage Reservations

• **Specification of the Use Case Browse Reservations**

Table 1.16: Use Case Specification: Browse Reservations

Title	Browse Reservations
Summary	Engineer views all their reservations for USB devices, cameras, and VM resources, and inspects the full details of any individual reservation
Actor	Engineer
Precondition	The engineer is authenticated and has created at least one reservation
Post-condition	Engineer has viewed their reservation list and the complete details of a selected reservation including dates, status, resource type, and any pending actions
Basic Scenario	1. Engineer navigates to the reservations section; system retrieves and displays all reservations with resource type, date range, and status; engineer may filter by resource type or status 2. Engineer selects a reservation; system displays the full detail view showing resource type, date range, status, special requirements, approval status, and any available actions
Error Scenario	1. If no reservations exist, system displays an empty state with an option to create one 2. If a selected reservation has been deleted, system shows a not-found notice and returns to the list

• **Specification of the Use Case Manage Reservation Lifecycle**

Table 1.17: Use Case Specification: Manage Reservation Lifecycle

Title	Manage Reservation Lifecycle
Summary	Engineer consults the availability calendar, creates new reservations for USB devices, cameras, or VM resources, and cancels existing ones when no longer needed
Actor	Engineer
Precondition	The engineer is authenticated and has access to the reservation system; the desired resource must be available for new reservations
Post-condition	A reservation is created and pending approval, or an existing reservation is cancelled and the resource is released for other users
Basic Scenario	1. Engineer opens the reservation calendar; system displays a calendar view with resource availability and existing reservations shown as blocked time slots; engineer may navigate between dates to find a suitable slot 2. For **Create**: engineer selects an available time slot and resource type, system checks for scheduling conflicts, engineer submits the request, system creates the reservation with pending status and confirms the submission with approval status 3. For **Cancel**: engineer selects an existing reservation and confirms cancellation; system cancels the record, releases the resource, and makes it available for other reservations
Error Scenario	1. If calendar data fails to load, system shows an error with a retry option 2. If the selected time slot conflicts with an existing reservation, system shows alternative availability 3. If the resource is not available, system displays availability information 4. If the reservation has already started, cancellation may be restricted 5. If cancellation fails, system logs the error and preserves the reservation

3.2.2.3 Refinement of the Use Case Manage Virtual Lab

Figure 1.7 presents the refinement of the Use Case Manage Virtual Lab for the Engineer actor. This use case includes the following sub-use cases:

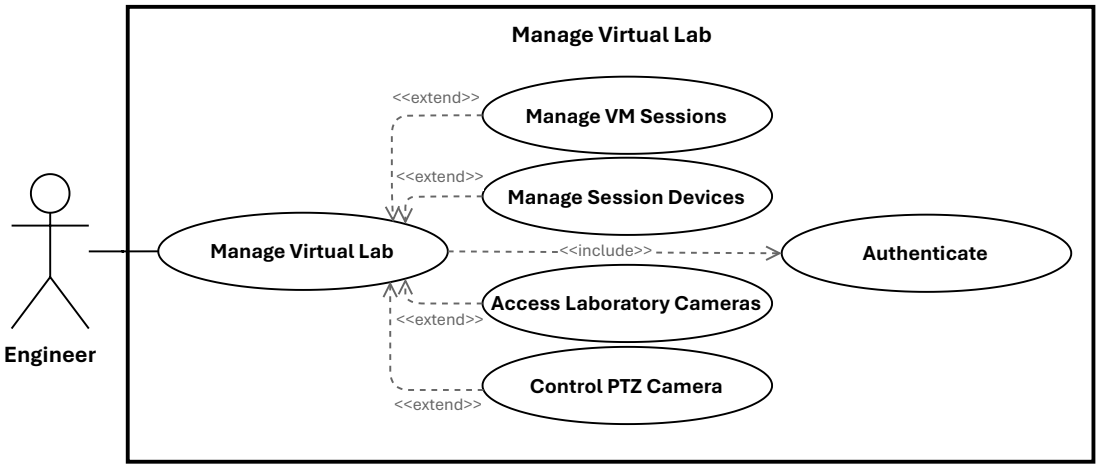

Figure 1.7: Use Case Manage Virtual Lab

• **Specification of the Use Case Manage VM Sessions**

Table 1.18: Use Case Specification: Manage VM Sessions

Title	Manage VM Sessions
Summary	Engineer views, provisions, extends, and terminates virtual machine sessions to access and manage their remote laboratory environment
Actor	Engineer (and Administrator when authorized)
Precondition	The engineer is authenticated, verified, and authorized through the `provision-vm` ability where applicable
Post-condition	The VM session is created, extended, or terminated and infrastructure resources are allocated or released accordingly
Basic Scenario	1. Engineer opens the VM sessions interface; system displays all sessions with status (active, expired, terminated), creation time, and remaining duration; engineer may filter by status or date range 2. For **Provision**: engineer submits a request to create a new session; system validates authorization and rate-limit constraints, selects suitable server and node resources, provisions the session record with associated infrastructure metadata, and returns the created session to the interface 3. For **Extend**: engineer selects an active session and chooses an extension duration; system validates eligibility and rate limits, updates the expiration time, and confirms the new deadline 4. For **Terminate**: engineer confirms the termination of an active session; system validates the request, releases associated infrastructure resources, and marks the session as terminated
Error Scenario	1. If no sessions exist, system displays an empty state with an option to provision one 2. Authorization failure or rate-limit excess prevents provisioning and surfaces an explanatory error 3. If no infrastructure capacity is available, system returns a controlled failure 4. Extension is blocked if the session has expired or the maximum extension limit is reached 5. If termination fails, system logs the error and preserves the session state 6. Provisioning or extension failures preserve traceability through logs or alerts

- **Specification of the Use Case Manage Session Devices**

Table 1.19: Use Case Specification: Manage Session Devices

Title	Manage Session Devices
Summary	Engineer attaches and detaches USB devices to and from an active VM session, and joins a waiting queue when a desired device is currently in use
Actor	Engineer
Precondition	The engineer has an active VM session
Post-condition	The device is attached to or detached from the session, or the engineer is queued and notified when the device becomes available
Basic Scenario	1. Engineer views their active session; system displays available devices with availability status 2. For **Attach**: engineer selects an available device; system validates compatibility and availability, establishes the connection between the device and the VM, and confirms attachment; the device becomes accessible from within the VM environment 3. For **Detach**: engineer selects an attached device and confirms detachment; system disconnects the device, releases it for use by other sessions, and confirms the action 4. For **Queue**: if the desired device is in use, engineer selects the option to join the waiting queue; system adds the engineer with their queue position and estimated wait time, and notifies them when the device becomes available
Error Scenario	1. If the device is already in use, system displays availability information and offers the queue option 2. If the device is incompatible with the VM, system prevents attachment 3. If attachment or detachment fails, system logs the error and allows retry 4. If detachment is requested while a process is actively using the device, system warns or delays the action 5. If the engineer is already in the queue, system displays their current position 6. If the queue is full, system prevents joining and displays capacity information

• **Specification of the Use Case Access Laboratory Cameras**

Table 1.20: Use Case Specification: Access Laboratory Cameras

Title	Access Laboratory Cameras
Summary	Engineer browses available laboratory cameras, views their live streams, and adjusts stream resolution to monitor laboratory equipment during an active VM session
Actor	Engineer
Precondition	The engineer has an active VM session and at least one camera is configured for the laboratory
Post-condition	Engineer is viewing a live camera stream at the selected resolution
Basic Scenario	1. Engineer opens the camera panel for their active session; system displays available cameras with status (available, in use, offline) and capabilities (PTZ support, resolution options) 2. Engineer selects a camera; system displays available resolution options with details (width, height, frame rate, bandwidth requirements) 3. Engineer selects a resolution; system initiates the stream connection and displays the live video feed in a player window 4. Engineer may switch resolution at any time while viewing; system reconfigures the stream and updates bandwidth requirements accordingly 5. Stream continues until the engineer stops viewing or the session ends
Error Scenario	1. If no cameras are available or configured, system displays an empty state 2. If camera status cannot be retrieved, system shows an error with a retry option 3. If a camera is offline, system displays an unavailable message 4. If the stream fails to load, system offers retry and may suggest a lower resolution 5. If resolution switching fails, system preserves the current resolution

• **Specification of the Use Case Control PTZ Camera**

Table 1.21: Use Case Specification: Control PTZ Camera

Title	Control PTZ Camera
Summary	Engineer acquires exclusive control over a PTZ camera, sends pan, tilt, and zoom commands to adjust its view, and releases control when finished
Actor	Engineer
Precondition	The engineer has an active VM session and is viewing a camera that supports PTZ control
Post-condition	Camera control is acquired, commands are applied, and control is released and made available to other users
Basic Scenario	1. Engineer views a PTZ-capable camera stream; system displays control availability status 2. Engineer requests camera control; system checks availability and if free grants exclusive control to the engineer, updates control status, and notifies other users 3. Engineer uses the PTZ control interface to send pan (horizontal), tilt (vertical), and zoom commands; system forwards each command to the camera, which moves to the new position; stream updates to reflect the new view 4. When finished, engineer releases control; system revokes exclusive access, marks the camera as available, and notifies other users
Error Scenario	1. If the camera does not support PTZ, system disables control options 2. If the camera is already controlled by another user, system displays a queue option 3. If a PTZ command is out of range or invalid, system ignores or limits it 4. If the camera does not respond to a command, system displays an error 5. If control release fails, system logs the error and preserves the current control state

3.2.2.4 Refinement of the Use Case Handle Payments

Figure 1.8 presents the refinement of the Use Case Handle Payments for the Engineer actor. This use case includes the following sub-use cases:

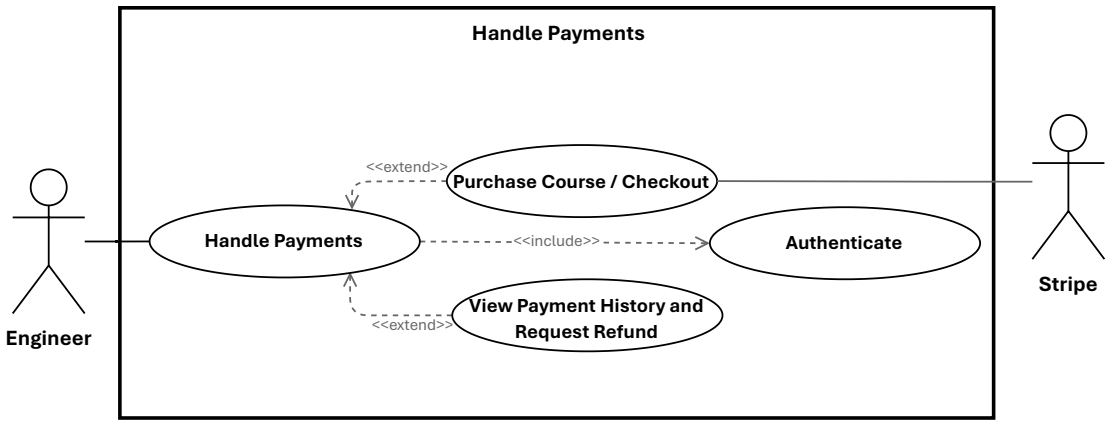

Figure 1.8: Use Case Handle Payments

• **Specification of the Use Case Purchase Course / Checkout**

Table 1.22: Use Case Specification: Purchase Course / Checkout

Title	Purchase Course / Checkout
Summary	Engineer purchases a paid training path through Stripe checkout and is directed to a confirmation or cancellation screen depending on the outcome
Primary Actor	Engineer
Secondary Actor	Stripe
Precondition	The engineer has selected a training path for enrollment and has a valid payment method
Post-condition	Payment is confirmed and enrollment is activated, or the checkout is cancelled and no enrollment or payment record is created
Basic Scenario	1. Engineer reviews the training path details and price, then initiates checkout 2. System redirects the engineer to the Stripe payment page 3. For **Success**: engineer enters payment details and confirms; Stripe processes the payment and sends a webhook confirmation; system activates the enrollment, generates a receipt, and displays a confirmation screen with transaction details and a link to the training path 4. For **Cancellation**: engineer aborts on the Stripe page; Stripe redirects to the platform cancellation screen; system displays a cancellation notice and offers options to retry checkout or return to the training path details; no payment or enrollment record is created
Error Scenario	1. If the payment is declined, system displays the rejection reason 2. If the webhook is not received, system polls Stripe for status 3. If the training path price changed during checkout, system requests re-confirmation 4. If enrollment activation is delayed after successful payment, system shows a processing message 5. If the Stripe redirect fails in either direction, engineer may need to manually navigate back to the platform

• **Specification of the Use Case View Payment History and Request Refund**

Table 1.23: Use Case Specification: View Payment History and Request Refund

Title	View Payment History and Request Refund
Summary	Engineer reviews past payment transactions and submits a refund request for an eligible purchase
Actor	Engineer
Precondition	The engineer is authenticated and has at least one completed payment transaction
Post-condition	Engineer has reviewed their transaction history, or a refund request is created with pending status and submitted for administrative review
Basic Scenario	1. Engineer navigates to the payment history section; system displays all transactions with date, training path, amount, and status; engineer may filter by date range or status 2. Engineer selects a transaction to view its details 3. For **Refund**: engineer clicks the refund option on an eligible transaction; system displays refund eligibility and a request form; engineer provides a reason and submits; system creates a pending refund request, notifies administrators for review, and allows the engineer to track the request status
Error Scenario	1. If no payment history exists, system displays an empty state 2. If the selected transaction is not eligible for a refund, system displays the refund policy 3. If a refund request already exists for the transaction, system shows its current status

3.2.2.5 Refinement of the Use Case Manage Account

Figure 1.9 presents the refinement of the Use Case Manage Account for the Engineer actor. This use case includes the following sub-use cases:

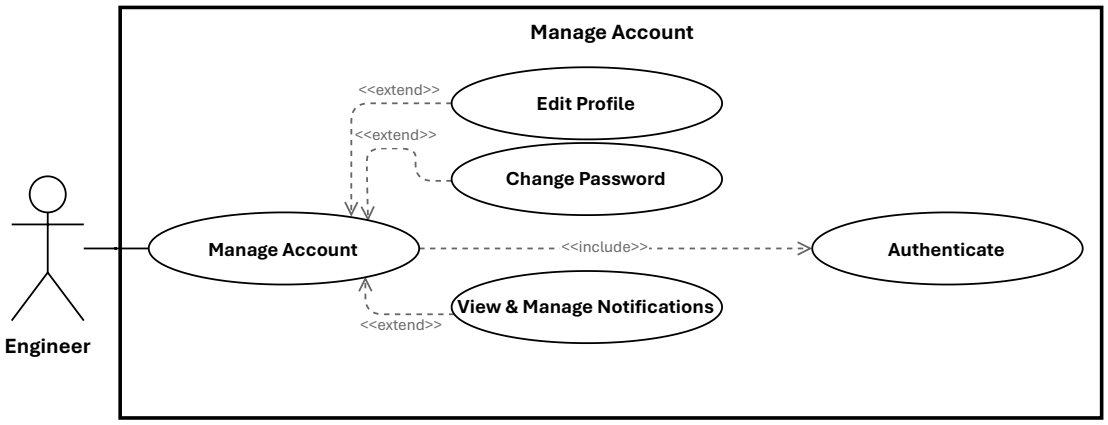

Figure 1.9: Use Case Manage Account

• **Specification of the Use Case Edit Profile**

Table 1.24: Use Case Specification: Edit Profile

Title	Edit Profile
Summary	Engineer updates their personal profile information including name, bio, and preferences
Actor	Engineer
Precondition	The engineer is authenticated
Post-condition	Profile information is updated and persisted
Basic Scenario	1. Engineer navigates to profile settings 2. System displays current profile information 3. Engineer updates name, bio, or other profile fields 4. System validates the updated information 5. System persists the changes 6. Profile is updated across the platform
Error Scenario	1. If validation fails, the system displays specific error messages 2. If update fails, the system preserves the previous profile data

• **Specification of the Use Case Change Password**

Table 1.25: Use Case Specification: Change Password

Title	Change Password
Summary	Engineer changes their account password for security purposes
Actor	Engineer
Precondition	The engineer is authenticated and knows their current password
Post-condition	The password is updated and the engineer must authenticate with the new password
Basic Scenario	1. Engineer navigates to password settings 2. System displays password change form 3. Engineer enters current password 4. Engineer enters new password and confirms it 5. System validates current password and new password requirements 6. System updates the password hash 7. System confirms the change and may require re-authentication
Error Scenario	1. If current password is incorrect, the system rejects the change 2. If new password does not meet requirements, the system displays validation errors 3. If passwords do not match, the system prompts for confirmation

• **Specification of the Use Case View & Manage Notifications**

Table 1.26: Use Case Specification: View & Manage Notifications

Title	View & Manage Notifications
Summary	Engineer views system notifications and manages their read/unread status
Actor	Engineer
Precondition	The engineer is authenticated and has received notifications
Post-condition	Engineer views notifications and can mark them as read or take actions
Basic Scenario	1. Engineer navigates to notifications section 2. System displays list of notifications with read/unread status 3. System shows notification type, content, and timestamp 4. Engineer can click a notification to view details 5. Engineer can mark notifications as read or unread 6. Engineer can dismiss or delete notifications
Error Scenario	1. If no notifications exist, the system displays an empty-state message

3.2.3 Teacher Use Cases

The Teacher actor is centered on pedagogical production and publication workflows. The most relevant use cases are training path creation, content maintenance, quiz publication, and submission for approval.

• **Specification of the Use Case View Analytics & Performance**

Table 1.27: Use Case Specification: View Analytics & Performance

Title	View Analytics & Performance
Summary	Teacher reviews enrollment statistics, completion rates, and learner performance for their training paths
Actor	Teacher
Precondition	The teacher is authenticated and has at least one published training path
Post-condition	The teacher views analytics data for their training paths
Basic Scenario	1. Teacher opens the teaching analytics dashboard 2. System retrieves enrollment counts, completion rates, and average quiz scores per training path 3. System displays charts and tables with time-series trends 4. Teacher may filter analytics by date range or training path 5. Teacher may drill down into individual module or unit performance
Error Scenario	1. If no training paths exist, the system displays an empty-state message 2. If analytics data is still computing, the system shows a loading indicator

3.2.3.1 Refinement of the Use Case Manage Training Paths

Figure 1.10 presents the refinement of the Use Case Manage Training Paths for the Teacher actor. This use case includes the following sub-use cases:

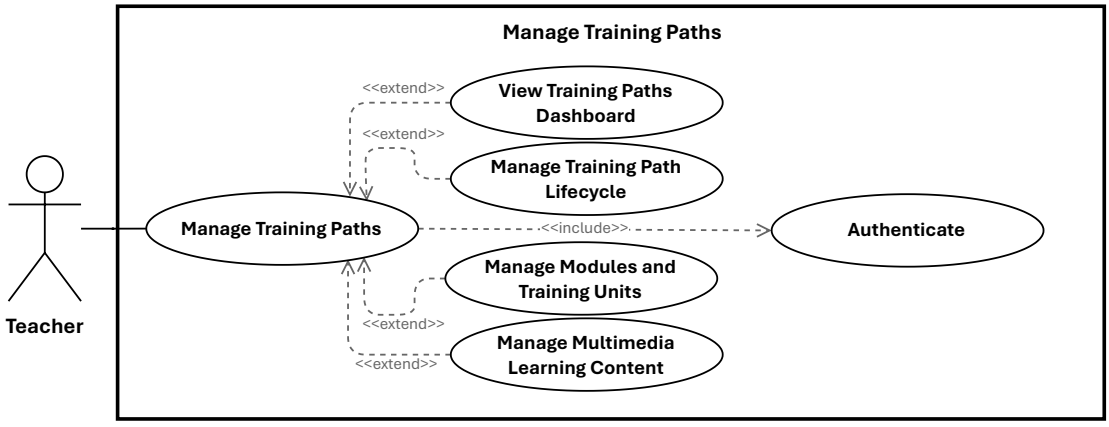

Figure 1.10: Use Case Manage Training Paths

• **Specification of the Use Case View Training Paths Dashboard**

Table 1.28: Use Case Specification: View Training Paths Dashboard

Title	View Training Paths Dashboard
Summary	Teacher views all owned training paths with statistics including total engineers, average ratings, and completion rates
Actor	Teacher
Precondition	Teacher is authenticated, verified, and approved to teach
Post-condition	Teacher views the list of owned training paths with key metrics
Basic Scenario	1. Teacher navigates to the teaching dashboard 2. System retrieves all training paths owned by the teacher 3. System calculates and displays statistics per path 4. Teacher may click any training path to view or edit details
Error Scenario	1. If no training paths exist, system displays empty state with create option 2. If teacher is not approved, system redirects to pending approval page

• **Specification of the Use Case Manage Training Path Lifecycle**

Table 1.29: Use Case Specification: Manage Training Path Lifecycle

Title	Manage Training Path Lifecycle
Summary	Teacher creates, edits, deletes, archives, restores, and submits training paths for review throughout their lifecycle
Actor	Teacher
Precondition	Teacher is authenticated, verified, approved to teach, and owns the target training path where applicable
Post-condition	The training path is created, updated, removed, archived, restored, or submitted for review according to the action performed
Basic Scenario	1. Teacher selects a lifecycle action from the dashboard: create, edit, delete, archive, restore, or submit for review 2. For **Create**: Teacher fills in title, description, category, price, and difficulty, then submits; system creates the path with draft status 3. For **Edit**: Teacher modifies desired metadata fields and submits; system validates and persists changes 4. For **Delete**: Teacher confirms permanent deletion; system cascades removal of all associated content 5. For **Archive**: Teacher confirms archiving; system soft-deletes the path and removes it from public listings 6. For **Restore**: Teacher selects an archived path and confirms restoration; system returns it to its previous state 7. For **Submit for Review**: Teacher confirms submission; system transitions the path to pending review and notifies administrators
Error Scenario	1. Missing or invalid fields cause validation failure and surface specific error messages 2. Deletion is blocked if active enrollments exist 3. Submission is rejected if the training path is incomplete 4. Unauthorized actions are denied and access is blocked 5. Persistence failures preserve the previous state and surface a recoverable error

• **Specification of the Use Case Manage Modules and Training Units**

Table 1.30: Use Case Specification: Manage Modules and Training Units

Title	Manage Modules and Training Units
Summary	Teacher adds, edits, reorders, and deletes modules and their contained training units to structure the learning content of a training path
Actor	Teacher
Precondition	Teacher is authorized to edit the target training path
Post-condition	The training path structure is updated to reflect the added, modified, reordered, or removed modules and training units
Basic Scenario	1. Teacher opens the training path editor 2. Teacher selects an action on a module or training unit: add, edit, reorder, or delete 3. For **Add**: Teacher enters title and description and submits; system creates the item and appends it to the structure 4. For **Edit**: Teacher modifies desired fields and submits; system validates and persists updates 5. For **Reorder**: Teacher drags items to the desired position; system persists the new order 6. For **Delete**: Teacher confirms deletion; system removes the item and all contained content with a cascading delete 7. System reflects all structural changes in the editor view
Error Scenario	1. Missing required fields surface validation errors 2. Deletion of items with active engineer progress is warned or blocked 3. Unauthorized access is denied 4. Persistence failures preserve the previous structure and surface a recoverable error

• **Specification of the Use Case Manage Multimedia Learning Content**

Table 1.31: Use Case Specification: Manage Multimedia Learning Content

Title	Manage Multimedia Learning Content
Summary	Teacher creates or updates articles, videos, captions, and quizzes associated with a training unit
Actor	Teacher
Precondition	Teacher is authorized to edit the target training path and training unit
Post-condition	The selected educational asset is created, updated, published, unpublished, or deleted according to the action performed
Basic Scenario	1. Teacher selects a training unit and opens the content editor 2. For **Article**: Teacher writes or edits rich text content with formatting and media; system persists on save 3. For **Video**: Teacher uploads a video file; system validates format and size, stores the file, and queues it for transcoding; Teacher may monitor transcoding status, retry on failure, and manage caption files per language 4. For **Quiz**: Teacher creates a quiz with title, passing score, and time limit; adds, reorders, and manages multiple-choice questions with marked correct answers; then publishes or unpublishes the quiz; Teacher may also view attempt statistics including pass rates and average scores 5. System validates content structure and ownership at each step 6. System persists the asset and updates the training unit status accordingly
Error Scenario	1. Invalid or incomplete content causes validation failure with specific messages 2. Unauthorized editing is blocked 3. Video format or size violations prevent upload 4. Media processing failure leaves the asset in a retryable state 5. Quiz publication is blocked if no questions exist

3.2.3.2 Refinement of the Use Case Manage Forum Interactions

Figure 1.11 presents the refinement of the Use Case Manage Forum Interactions for the Teacher actor. This use case includes the following sub-use cases:

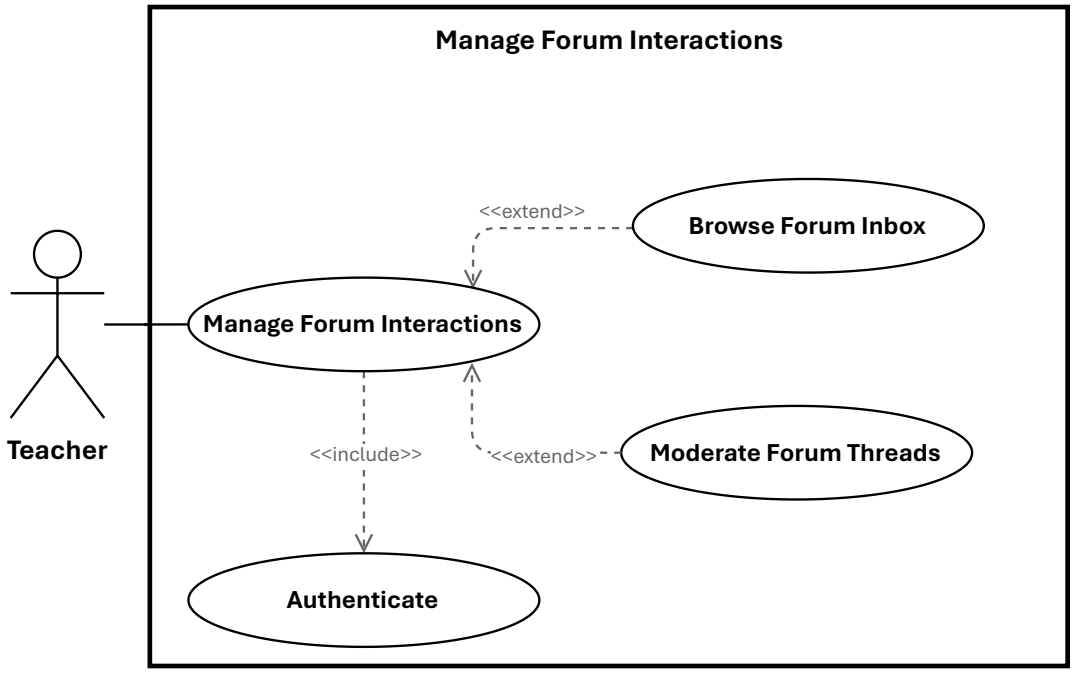

Figure 1.11: Use Case Manage Forum Interactions

• **Specification of the Use Case Browse Forum Inbox**

Table 1.32: Use Case Specification: Browse Forum Inbox

Title	Browse Forum Inbox
Summary	Teacher opens the discussion forum inbox and filters threads by status to identify questions requiring attention
Actor	Teacher
Precondition	Teacher owns at least one training path with forum activity
Post-condition	Teacher views a filtered list of threads and may proceed to read or respond
Basic Scenario	1. Teacher navigates to the forum inbox from the teaching dashboard 2. System displays threads from all owned training paths with reply counts and timestamps 3. Teacher selects a filter to narrow the view: • **Flagged** — shows threads or replies that have been reported and require moderation • **Unanswered** — shows threads with no teacher response yet • **Recent** — shows threads sorted by most recent activity 4. System retrieves and displays threads matching the selected filter 5. Teacher may select any thread to read, respond, or take a moderation action
Error Scenario	1. If no threads match the active filter, system displays an empty state 2. If filter retrieval fails, system falls back to showing all threads with an error notice 3. If teacher has no training paths, system displays an empty state with setup instructions

• **Specification of the Use Case Moderate Forum Threads**

Table 1.33: Use Case Specification: Moderate Forum Threads

Title	Moderate Forum Threads
Summary	Teacher moderates discussion threads within owned training paths by pinning, locking, resolving flags, and editing or removing inappropriate content
Actor	Teacher
Precondition	Teacher owns the training path containing the target thread or reply
Post-condition	Thread state is updated according to the moderation action performed; a log entry is created and affected engineers are notified where applicable
Basic Scenario	1. Teacher opens a thread from the forum inbox 2. Teacher selects a moderation action: • **Pin / Unpin** — promotes the thread to the top of the forum listing or returns it to chronological order • **Lock / Unlock** — prevents new replies from engineers or reopens the thread for discussion • **Resolve Flag** — teacher reviews flagged content, takes appropriate action (approve, edit, or delete), then marks the flag as resolved • **Edit / Delete post** — teacher corrects or removes inappropriate content and system logs the action with timestamp and reason 3. System validates ownership and applies the requested state change 4. System confirms the action and notifies affected engineers where applicable
Error Scenario	1. Unauthorized access is denied if teacher does not own the training path 2. If the target thread or post has already been removed, system displays a notice 3. Resolving a flag on non-flagged content surfaces an explanatory error 4. Persistence failures preserve the previous state and surface a recoverable error

3.2.3.3 Refinement of the Use Case Manage VM Assignments

Figure **??** presents the refinement of the Use Case Manage VM Assignments for the Teacher actor. This use case includes the following sub-use cases: • **Specification of the Use Case Submit VM Assignment**

Table 1.34: Use Case Specification: Submit VM Assignment

Title	Submit VM Assignment
Summary	Teacher creates a VM assignment request for a training unit, specifying template, duration, and justification
Actor	Teacher
Precondition	Teacher is authorized to request VMs for the target training unit
Post-condition	A new VM assignment record is created with pending status and notifications are dispatched as configured
Basic Scenario	1. Teacher opens the VM assignment form for a training unit 2. Teacher selects a VM template and duration, and provides a justification 3. System validates ownership and field values, then creates the assignment record 4. System confirms submission and displays the new assignment with its pending status
Error Scenario	1. Validation failures surface field-level errors and preserve input 2. If quota or node availability prevents assignment, system displays an explanatory error

• **Specification of the Use Case Manage Assignments**

Table 1.35: Use Case Specification: Manage VM Assignments

Title	Manage VM Assignments
Summary	Teacher views and cancels VM assignment requests associated with their training paths
Actor	Teacher
Precondition	Teacher is authenticated and associated with at least one training path
Post-condition	Teacher views current assignments or a selected assignment is cancelled and any scheduled provisioning is aborted
Basic Scenario	1. Teacher opens the VM assignments view 2. System displays assignments grouped by training path with status, template, and timestamps 3. Teacher may filter or sort the list, or open an assignment for details 4. To cancel, teacher selects an assignment and confirms the delete action 5. System verifies the assignment is in a cancellable state, removes or marks it cancelled, and updates the list
Error Scenario	1. If no assignments exist, system displays an empty state with setup instructions 2. If the assignment is already provisioned or in a non-cancellable state, system displays an explanatory error 3. Persistence failures preserve current state and surface a recoverable error

• **Specification of the Use Case Manage Earnings & Payouts**

Table 1.36: Use Case Specification: Manage Earnings & Payouts

Title	Manage Earnings & Payouts
Summary	Teacher views their earnings summary, requests a payout of available balance, and exports transaction history as a CSV report
Actor	Teacher
Precondition	Teacher is authenticated and has at least one training path with paid enrollments
Post-condition	Teacher has viewed financial data, submitted a payout request, or downloaded an earnings report according to the action performed
Basic Scenario	1. Teacher navigates to the earnings section 2. System displays total earnings, available balance, pending payouts, and a revenue breakdown by training path; teacher may filter by date range 3. For **Request Payout**: teacher enters the desired amount and selects a payment method; system validates the amount against the available balance and minimum threshold, creates a pending payout request, and confirms submission 4. For **Export Report**: teacher specifies a date range and clicks export; system generates a CSV file of transaction details and initiates the download
Error Scenario	1. If no earnings data exists, system displays an empty state 2. Payout amount exceeding available balance or falling below the minimum threshold surfaces a specific validation error 3. Invalid payment method prevents payout submission 4. An invalid or empty date range prevents CSV export 5. Service or export failures surface a recoverable error with a retry option

3.2.4 Administrator Use Cases

The Administrator actor coordinates governance, operations, and infrastructure. The administrator's responsibilities span nine major functional areas: dashboard and platform analytics, user and role management, infrastructure and hardware management, training

path management, reservation management, VM assignment approval, payout and refund processing, maintenance mode configuration, and forum moderation.

3.2.4.1 Refinement of the Use Case Consult Dashboard & Platform Analytics

Figure 1.12 presents the refinement of the Use Case Consult Dashboard & Platform Analytics for the Administrator actor. This use case includes the following sub-use cases:

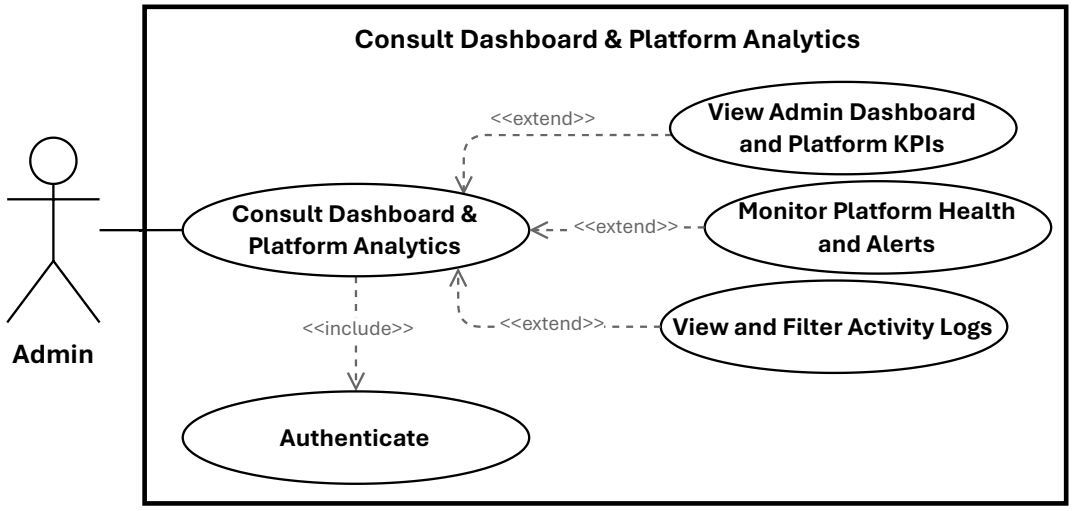

Figure 1.12: Use Case Consult Dashboard & Platform Analytics

• **Specification of the Use Case View Admin Dashboard and Platform KPIs**

Table 1.37: Use Case Specification: View Admin Dashboard and Platform KPIs

Title	View Admin Dashboard and Platform KPIs
Summary	Administrator views system-wide key performance indicators including user statistics, revenue, active sessions, and infrastructure health, with the ability to drill down into detailed breakdowns per metric category
Actor	Administrator
Precondition	The administrator is authenticated with admin privileges
Post-condition	The administrator has viewed current system KPIs, trend charts, and infrastructure status at the desired level of detail
Basic Scenario	1. Administrator opens the admin dashboard; system aggregates and displays real-time KPI cards covering active VM sessions, registered users, pending approvals, revenue, and infrastructure status indicators 2. Administrator may filter the dashboard by date range or metric category 3. Administrator drills down into a KPI card to access the detailed platform metrics view, which displays time-series charts and comparative breakdowns across user statistics (total, active, new registrations), revenue data (total, monthly, payout amounts), and session data (active sessions, duration, resource utilization) 4. Administrator may further filter detailed metrics by date range, user role, or revenue category
Error Scenario	1. If the metrics or KPI service is unavailable, system displays cached data with a staleness warning 2. If no data exists for the selected period, system shows an empty state message

• **Specification of the Use Case Monitor Platform Health and Alerts**

Table 1.38: Use Case Specification: Monitor Platform Health and Alerts

Title	Monitor Platform Health and Alerts
Summary	Administrator reviews the overall health status of platform services, inspects active system alerts with their severity and context, and acknowledges or resolves alerts to track response actions
Actor	Administrator
Precondition	The administrator is authenticated with admin privileges
Post-condition	The administrator has reviewed service health and alert status; any addressed alerts are updated to acknowledged or resolved and recorded accordingly
Basic Scenario	1. Administrator navigates to the platform health section; system displays uptime, error rates, and response times for each service (database, cache, queue, external APIs), highlighting any critical or warning conditions 2. Administrator may drill down into individual service health details and configure health check thresholds and alert rules 3. Administrator opens the system alerts panel; system displays a prioritized list of active alerts with severity levels; administrator selects an alert to view detailed context including timestamp, severity, affected resources, and suggested actions; administrator may filter by severity, type, or time range 4. For **Acknowledge / Resolve**: administrator selects an active alert, enters resolution notes or action taken, and confirms; system updates the alert status, records the resolution, and removes the alert from the active list; administrator may configure automatic alert resolution rules
Error Scenario	1. If the health monitoring service is unavailable, system displays a connection error with a retry option 2. If a service is unreachable, system marks it as critical and suggests investigation 3. If the alerting service is unreachable, system displays a connection error with a retry option 4. If an alert references a deleted resource, system marks it as stale and suggests dismissal 5. If an alert has already been resolved, system displays a notice 6. If a resolution update fails, system preserves the previous state and logs the error

• **Specification of the Use Case View and Filter Activity Logs**

Table 1.39: Use Case Specification: View and Filter Activity Logs

Title	View and Filter Activity Logs
Summary	Administrator views a summary of recent platform activity and browses the full paginated activity log, applying filters by date range, user, event type, or severity to locate specific events, with the option to export results for compliance or auditing
Actor	Administrator
Precondition	The administrator is authenticated with admin privileges and the activity logging system is active
Post-condition	The administrator has reviewed recent or filtered activity log entries and optionally exported the results
Basic Scenario	1. Administrator navigates to the activity section of the admin dashboard; system displays a summary of recent activity grouped by type (user actions, content updates, system events) with timestamps; administrator may filter by activity type or time range and click any entry for detailed information 2. Administrator navigates to the full Activity Logs view; system retrieves and displays the most recent log entries with pagination, showing timestamp, user, action, and affected resource per entry 3. Administrator applies filters (date range, user, event type, severity); system updates the display to match the criteria; administrator may save filter configurations for future use 4. Administrator exports filtered log entries to CSV or PDF for compliance or auditing purposes
Error Scenario	1. If no entries match the filter criteria, system displays an empty state with a descriptive message 2. If the filter query is too complex, system suggests simplifying the criteria 3. If log storage or the activity service is temporarily unavailable, system displays an error notification with a retry option

3.2.4.2 Refinement of the Use Case Manage Users & Roles

Figure 1.13 presents the refinement of the Use Case Manage Users & Roles for the Administrator actor. This use case includes the following sub-use cases:

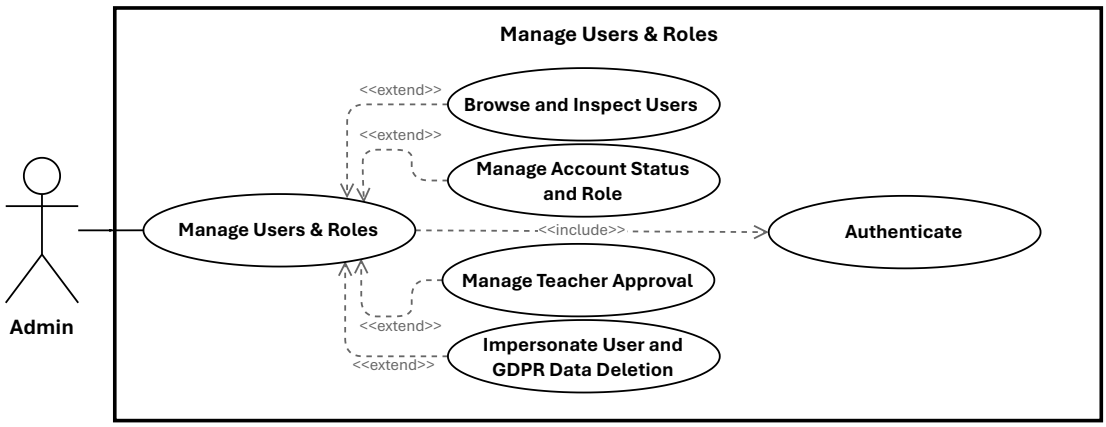

Figure 1.13: Use Case Manage Users & Roles

• **Specification of the Use Case Browse and Inspect Users**

Table 1.40: Use Case Specification: Browse and Inspect Users

Title	Browse and Inspect Users
Summary	Administrator browses the full list of registered users with filtering and sorting options, and inspects the detailed profile, activity history, and account status of any individual user
Actor	Administrator
Precondition	The administrator is authenticated with admin privileges
Post-condition	Administrator has viewed the user list and the comprehensive details of a selected user, and may proceed to perform account management actions
Basic Scenario	1. Administrator navigates to the User Management section; system displays a paginated list of all registered users with name, email, role, status, and registration date; administrator may filter by role or status, search by name or email, and sort by registration date, name, or activity 2. Administrator selects a user; system displays the full detail view with sections covering profile, account status, activity history, training path enrollments and progress, and payment history and refund requests 3. Administrator may proceed to perform account actions directly from the detail view
Error Scenario	1. If no users exist or the user service is unavailable, system displays an appropriate empty state or error notification 2. If the selected user does not exist, system displays a not-found error and returns to the list

• **Specification of the Use Case Manage Account Status and Role**

Table 1.41: Use Case Specification: Manage Account Status and Role

Title	Manage Account Status and Role
Summary	Administrator suspends or unsuspends a user account and changes a user's role, updating their platform access and permissions accordingly
Actor	Administrator
Precondition	The administrator is authenticated with admin privileges and has opened the target user's detail page
Post-condition	The user's account status or role is updated, all active sessions are terminated where applicable, and the user is notified of the change
Basic Scenario	1. Administrator selects an account action from the user detail page 2. For **Suspend**: administrator provides a reason and confirms; system updates the user's status to suspended, terminates all active sessions, and notifies the user; the user can no longer log in or access the platform 3. For **Unsuspend**: administrator confirms the restoration; system updates the user's status to active and notifies the user; the user can log in and access the platform again 4. For **Change Role**: administrator selects a new role (engineer, teacher, or admin) and confirms; system validates the change, updates the user's role and permissions, and notifies the user
Error Scenario	1. If the user is already in the target status, system displays an explanatory notice 2. A role change that would leave no administrators is blocked with an explanatory error 3. Persistence failures for any action preserve the current state and surface a recoverable error

• **Specification of the Use Case Manage Teacher Approval**

Table 1.42: Use Case Specification: Manage Teacher Approval

Title	Manage Teacher Approval
Summary	Administrator reviews and approves pending teacher registration applications, and revokes teacher privileges from approved accounts when required
Actor	Administrator
Precondition	The administrator is authenticated with admin privileges; at least one teacher account is pending approval or an approved teacher account exists for revocation
Post-condition	The teacher account status is updated to approved or revoked and the user is notified; their ability to create and submit training paths is granted or removed accordingly
Basic Scenario	1. For **Approve**: administrator opens the pending teacher accounts list; system displays each applicant's profile, credentials, and submitted documents; administrator reviews qualifications and approves the account; system grants teacher-level privileges and notifies the teacher, who can now create and submit training paths 2. For **Revoke**: administrator navigates to the teacher's user detail page and selects the revoke action; system requires a reason; administrator provides the reason and confirms; system downgrades the user's role, notifies the user, and prevents them from creating or submitting new training paths; existing published content remains accessible
Error Scenario	1. If the teacher's documents are incomplete during approval, administrator requests additional information 2. If the target user is not a teacher, revocation is blocked with an explanatory error 3. Persistence failures for either action preserve the current state and surface a recoverable error

• **Specification of the Use Case Impersonate User and GDPR Data Deletion**

Table 1.43: Use Case Specification: Impersonate User and GDPR Data Deletion

Title	Impersonate User and GDPR Data Deletion
Summary	Administrator either impersonates a user account to debug issues or view the platform from their perspective, or permanently deletes a user account and all associated data in compliance with GDPR requirements
Actor	Administrator
Precondition	The administrator is authenticated with admin privileges and the target user account exists
Post-condition	An impersonation session is created and logged for audit purposes, or the user account and all associated data are permanently and irreversibly deleted and the deletion is logged
Basic Scenario	1. Administrator navigates to the target user's detail page and selects a privileged action 2. For **Impersonate**: administrator confirms the impersonation; system creates an impersonation session, logs the action for audit purposes, and places the administrator in the target user's context; the administrator can now interact with the platform as that user 3. For **GDPR Deletion**: system displays a warning listing all data to be permanently removed (profile, enrollments, payments, activity logs); administrator confirms the deletion; system permanently removes the user account and all associated data and logs the action for audit purposes
Error Scenario	1. If the target user does not exist, system displays a not-found error for either action 2. If impersonation fails, system retains the current administrator session and surfaces an error 3. If the deletion fails, system preserves all user data and surfaces a recoverable error

3.2.4.3 Refinement of the Use Case Manage Infrastructure and Hardware

Figure 1.14 presents the refinement of the Use Case Manage Infrastructure and Hardware for the Administrator actor. This use case includes the following sub-use cases:

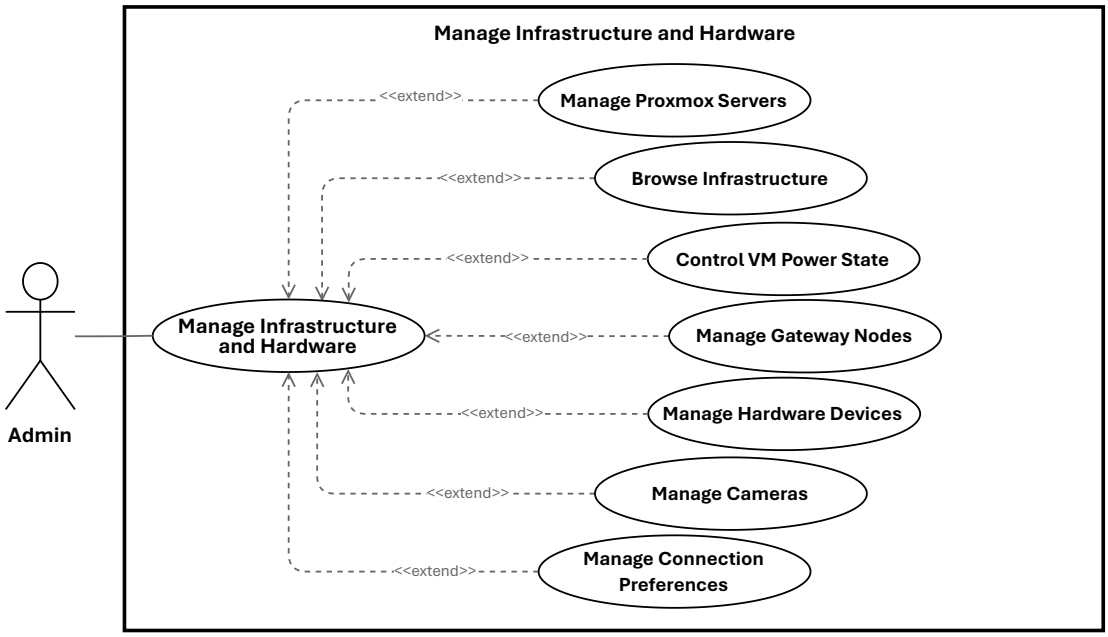

Figure 1.14: Use Case Manage Infrastructure and Hardware

- **Specification of the Use Case Manage Proxmox Servers**

Table 1.44: Use Case Specification: Manage Proxmox Servers

Title	Manage Proxmox Servers
Summary	Administrator registers, configures, tests, inactivates, deletes, and synchronizes Proxmox servers that provide the VM provisioning infrastructure
Actor	Administrator
Precondition	The administrator is authenticated with admin privileges
Post-condition	The Proxmox server registry is updated to reflect the registered, edited, inactivated, deleted, or synchronized server according to the action performed
Basic Scenario	1. Administrator navigates to the Proxmox Servers section; system displays all registered servers with name, host, status, and node count 2. For **Register**: administrator fills in server name, host, port, username, and password; system validates the connection to the Proxmox API, stores credentials securely, and adds the server to the infrastructure list 3. For **Edit Credentials**: administrator selects a server and updates connection details; system validates the new credentials by testing the connection before persisting the changes 4. For **Test Connection**: administrator clicks Test Connection on a selected server; system attempts to reach the Proxmox API and displays the result including server version and node information on success 5. For **Inactivate**: administrator confirms the action; system marks the server as inactive, preventing new VM provisioning while existing sessions continue normally 6. For **Delete**: system checks for active VMs; administrator confirms permanent deletion; system removes the server and all associated data and logs the action for audit purposes 7. For **Sync Nodes**: administrator clicks Sync Nodes on an active server; system connects to the Proxmox API, updates local node records with added or changed nodes, removes nodes that no longer exist, and displays a sync result summary
Error Scenario	1. If the server list cannot be retrieved, system shows an error notification 2. Connection test failures (timeout, authentication error, unreachable host) surface a specific error and prevent registration or credential updates from proceeding

• **Specification of the Use Case Browse Infrastructure: Nodes and VMs**

Table 1.45: Use Case Specification: Browse Infrastructure: Nodes and VMs

Title	Browse Infrastructure: Nodes and VMs
Summary	Administrator views all nodes across registered Proxmox servers, inspects the VMs running on a selected node, and monitors all currently running VMs platform-wide
Actor	Administrator
Precondition	The administrator is authenticated with admin privileges and at least one Proxmox server is registered
Post-condition	The administrator has viewed the node list, the VMs on a selected node, or the platform-wide list of running VMs with resource utilization details
Basic Scenario	1. Administrator navigates to the Nodes section; system displays all nodes from registered Proxmox servers with name, server, status, CPU, memory, and storage metrics; administrator may filter by server or status 2. Administrator selects a node; system retrieves and displays all VMs on that node with name, status, CPU, memory, and disk allocation; administrator may filter by status or resource type and view detailed VM configuration 3. Administrator navigates to the Running VMs section; system retrieves all VMs with running status across all nodes and displays them with name, node, owner, CPU, memory, and uptime; administrator may filter by node or owner
Error Scenario	1. If no nodes or VMs are available, system displays an appropriate empty state 2. If any list cannot be retrieved, system shows an error notification with a retry option

• **Specification of the Use Case Control VM Power State**

Table 1.46: Use Case Specification: Control VM Power State

Title	Control VM Power State
Summary	Administrator starts, stops, or reboots a virtual machine on a Proxmox node by sending the corresponding command to the Proxmox API
Actor	Administrator
Precondition	The administrator is authenticated and has selected a VM on a Proxmox node; the VM must be in the appropriate state for the requested action
Post-condition	The VM power state is updated and reflected in the infrastructure view
Basic Scenario	1. Administrator selects a VM from the node VM list and chooses a power action 2. For **Start**: system sends a start command to the Proxmox API; Proxmox starts the VM and system displays the updated status as running 3. For **Stop**: system sends a stop command to the Proxmox API; Proxmox stops the VM and system displays the updated status as stopped 4. For **Reboot**: system sends a reboot command to the Proxmox API; Proxmox reboots the VM and system displays the updated status as running after reboot
Error Scenario	1. If the VM is already in the target state, system displays an explanatory notice 2. If the VM is not running, reboot is blocked with an explanatory error 3. If any command fails, system displays an error and retains the current VM state

- **Specification of the Use Case Manage Gateway Nodes**

Table 1.47: Use Case Specification: Manage Gateway Nodes

Title	Manage Gateway Nodes
Summary	Administrator registers, edits, verifies, and deletes gateway nodes that handle hardware device management and USB forwarding
Actor	Administrator
Precondition	The administrator is authenticated with admin privileges
Post-condition	The gateway node registry is updated to reflect the registered, edited, verified, or deleted gateway according to the action performed
Basic Scenario	1. Administrator navigates to the Gateway Nodes section; system displays all registered gateways with name, host, status, and connected device count 2. For **Add**: administrator enters gateway name, host, port, and authentication details; system validates the connection, stores the configuration, and adds the gateway to the list 3. For **Edit**: administrator selects a gateway and updates its configuration; system validates the new settings by testing the connection before persisting the changes 4. For **Verify**: administrator clicks Verify on a selected gateway; system connects to the node, retrieves status and connected device count, and displays the verification result 5. For **Delete**: system checks for active devices; administrator confirms permanent deletion; system removes the gateway and all associated data and logs the action for audit purposes
Error Scenario	1. If the connection test fails during registration or edit, system displays an error and prevents the action from proceeding 2. A duplicate gateway name prompts for a unique name 3. If the gateway is unreachable during verification, system displays a connection error with diagnostic information 4. Deletion is blocked if active devices exist on the gateway 5. Persistence failures for any action retain the current state and surface a recoverable error

- **Specification of the Use Case Manage Hardware Devices**

Table 1.48: Use Case Specification: Manage Hardware Devices

Title	Manage Hardware Devices
Summary	Administrator discovers, monitors, assigns, and configures USB devices and cameras connected through gateway nodes, including VM assignment and device dedication settings
Actor	Administrator
Precondition	The administrator is authenticated with admin privileges and at least one active gateway node is registered
Post-condition	Hardware devices are discovered and registered, assigned to or unassigned from VMs, or configured as dedicated according to the action performed
Basic Scenario	1. Administrator navigates to the Hardware Devices section; system displays all registered devices (USB and cameras) with type, model, status, and gateway information; administrator may filter by type, status, or gateway 2. For **Discover**: administrator clicks Discover Devices; system queries all active gateways, retrieves device information (type, model, serial number, status), registers new devices, updates existing ones, and displays the discovery result 3. For **Assign USB to VM**: administrator selects a USB device and a target VM; system validates compatibility and updates the device configuration; the device becomes accessible from within the VM session 4. For **Unassign USB**: administrator selects an assigned USB device and confirms; system removes the assignment and releases the device for other VMs 5. For **Dedicated Device**: administrator selects a device, chooses dedication type (user or training path), and selects the target; system updates the dedication settings and confirms the configuration
Error Scenario	1. If no gateway nodes are available, discovery is blocked with an explanatory error 2. If discovery fails, system shows an error with diagnostic information 3. If a device is already assigned, system prompts to reassign or displays the current assignment 4. If the target VM is unavailable, assignment is blocked 5. If a device is in active use, dedication changes are prevented 6. Persistence failures for any action retain the current state

- **Specification of the Use Case Manage Cameras**

Table 1.49: Use Case Specification: Manage Cameras

Title	Manage Cameras
Summary	Administrator browses registered cameras, activates or deactivates them, and assigns or unassigns them to virtual machines individually or in bulk
Actor	Administrator
Precondition	The administrator is authenticated with admin privileges and at least one camera is registered
Post-condition	Camera status and VM assignment are updated according to the action performed
Basic Scenario	1. Administrator navigates to the Cameras section; system displays all registered cameras with name, status, PTZ support, and VM assignment; administrator may filter by status or PTZ capability 2. For **Activate / Deactivate**: administrator selects a camera and confirms the toggle; system updates the camera status and displays it as available or unavailable accordingly 3. For **Assign to VM**: administrator selects one or more cameras and a target VM; system validates all assignments, updates camera configurations, and confirms the result; assigned cameras become accessible from within the VM session 4. For **Unassign**: administrator selects an assigned camera and confirms; system removes the assignment and marks the camera as available
Error Scenario	1. If no cameras are registered, system displays an empty state 2. If a camera is already in the target activation state, system displays a notice 3. If a camera is already assigned, system prompts to reassign 4. If the target VM is unavailable, assignment is blocked 5. If a camera is not assigned, unassignment displays an explanatory notice 6. Persistence failures for any action retain the current state and surface a recoverable error

• **Specification of the Use Case Manage Connection Preferences**

Table 1.50: Use Case Specification: Manage Connection Preferences

Title	Manage Connection Preferences
Summary	Administrator configures preferred remote desktop connection settings such as protocol, resolution, color depth, and keyboard layout, which are applied to future Guacamole connection tokens
Actor	Administrator
Precondition	The administrator is authenticated with admin privileges
Post-condition	The connection preference profile is created or updated and will be applied to subsequent VM sessions
Basic Scenario	1. Administrator opens the connection preferences settings; system displays available protocols (RDP, VNC, SSH) and configurable parameters 2. Administrator selects a default protocol and configures resolution, color depth, and keyboard layout 3. Administrator saves the profile; system persists it and applies it to future Guacamole connection tokens 4. Administrator may create multiple profiles for different session types
Error Scenario	1. If the selected protocol is not supported, system rejects the configuration with an explanatory error 2. A duplicate profile name prompts for a unique name

3.2.4.4 Refinement of the Use Case Set Maintenance Mode

Figure 1.15 presents the refinement of the Use Case Set Maintenance Mode for the Administrator actor. This use case includes the following sub-use cases:

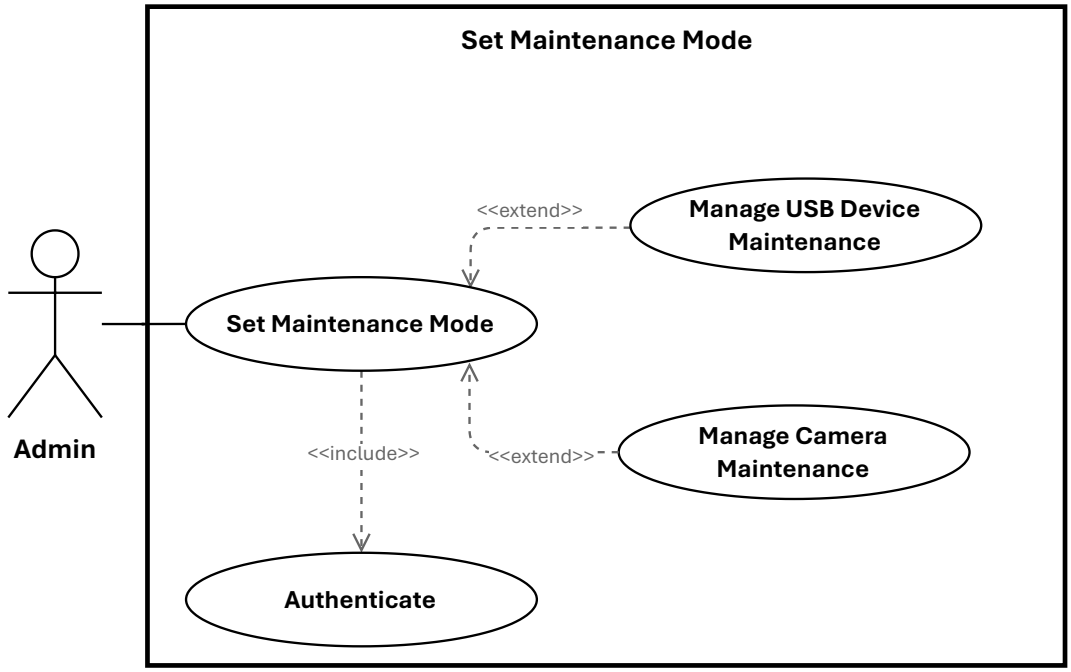

Figure 1.15: Use Case Set Maintenance Mode

- **Specification of the Use Case Manage USB Device Maintenance**

Table 1.51: Use Case Specification: Manage USB Device Maintenance

Title	Manage USB Device Maintenance
Summary	Administrator views all USB devices in maintenance, places available devices into maintenance mode with a reason and description, updates maintenance descriptions, and removes devices from maintenance to restore their availability
Actor	Administrator
Precondition	The administrator is authenticated with admin privileges; the target device must be in the appropriate state for the requested action
Post-condition	The USB device maintenance status and description are updated accordingly; devices in maintenance cannot accept new reservations and devices removed from maintenance become available again
Basic Scenario	1. Administrator navigates to the Maintenance section; system displays all devices currently in maintenance with type, model, maintenance reason, and start date; administrator may filter by device type or reason 2. For **Enable**: administrator selects an available USB device and clicks Enable Maintenance; system displays a form for the maintenance reason and description; administrator submits; system updates the device status to maintenance, prevents new reservations, and confirms the activation 3. For **Update Description**: administrator selects a device already in maintenance and clicks Edit Maintenance Description; administrator enters or updates the description; system saves the update and confirms 4. For **Disable**: administrator selects a device in maintenance and clicks Disable Maintenance; system confirms the action, updates the device status to available, clears the maintenance description, and confirms the deactivation
Error Scenario	1. If no devices are in maintenance, system displays an empty state 2. If the device list cannot be retrieved, system shows an error notification 3. If the device is already in the target maintenance state, system displays an explanatory notice 4. If a description update is attempted on a device not in maintenance, system displays an error 5. Persistence failures for any action retain the current state

• **Specification of the Use Case Manage Camera Maintenance**

Table 1.52: Use Case Specification: Manage Camera Maintenance

Title	Manage Camera Maintenance
Summary	Administrator places cameras into maintenance mode to prevent streaming and assignment, and removes them from maintenance to restore availability
Actor	Administrator
Precondition	The administrator is authenticated with admin privileges; the target camera must be in the appropriate state for the requested action
Post-condition	The camera maintenance status is updated; cameras in maintenance cannot be streamed or assigned, and cameras removed from maintenance become available again
Basic Scenario	1. Administrator selects a camera from the device list and chooses a maintenance action 2. For **Enable**: administrator clicks Enable Maintenance; system displays a form for the maintenance reason and description; administrator submits; system updates the camera status to maintenance, prevents streaming and assignment, and confirms the activation 3. For **Disable**: administrator selects a camera in maintenance and clicks Disable Maintenance; system confirms the action, updates the camera status to available, clears the maintenance description, and confirms the deactivation
Error Scenario	1. If the camera is already in the target maintenance state, system displays an explanatory notice 2. Persistence failures for any action retain the current state and surface a recoverable error

• **Specification of the Use Case Moderate Forum Threads**

Table 1.53: Use Case Specification: Moderate Forum Threads

Title	Moderate Forum Threads
Summary	Administrator reviews flagged forum threads and replies reported by users, and dismisses flags on content that does not warrant removal, restoring it to normal status
Actor	Administrator
Precondition	The administrator is authenticated with admin privileges
Post-condition	Reviewed content is either unflagged and removed from the moderation queue, or retained pending further action; all moderation decisions are logged for audit purposes
Basic Scenario	1. Administrator navigates to the Forum Moderation section; system displays a prioritized list of flagged threads and replies with flag reason, reporter, and post content; administrator may filter by severity, type, or time range 2. Administrator selects a flagged item and reviews its full thread context and flag reason 3. If the content does not warrant action, administrator selects Unflag; system removes the flag from the thread or reply, clears it from the moderation queue, and logs the decision for audit purposes
Error Scenario	1. If no flagged posts exist, system displays an empty state 2. If the forum service is unavailable, system displays an error notification 3. If the selected item has already been unflagged, system displays an explanatory notice 4. If the unflag action fails, system retains the current flag state and surfaces a recoverable error

3.2.4.5 Refinement of the Use Case Manage Training Paths

Figure 1.16 presents the refinement of the Use Case Manage Training Paths for the Administrator actor. This use case includes the following sub-use cases:

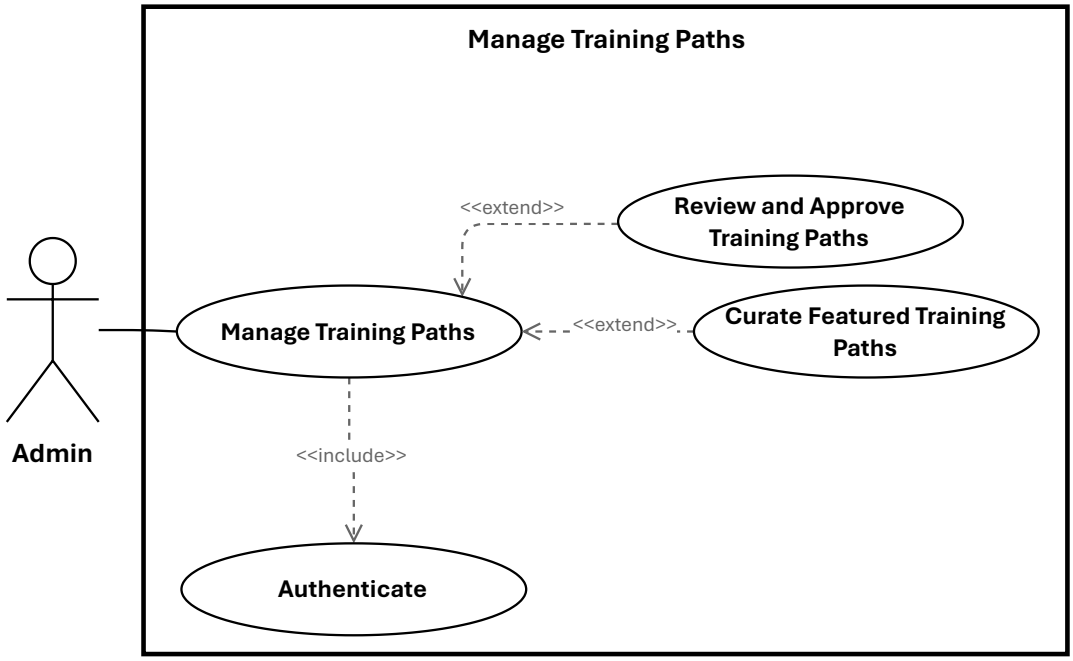

Figure 1.16: Use Case Manage Training Paths

- **Specification of the Use Case Review and Approve Training Paths**

Table 1.54: Use Case Specification: Review and Approve Training Paths

Title	Review and Approve Training Paths
Summary	Administrator browses all training paths, reviews submitted ones pending approval, and approves or rejects them with feedback to control what is published on the platform
Actor	Administrator
Precondition	The administrator is authenticated and authorized through the admin-only policy; at least one training path exists or is pending review
Post-condition	The training path status is updated to approved or rejected; approved paths become available in public listings and the author is notified in either case
Basic Scenario	1. Administrator navigates to the Training Paths section; system displays all training paths with title, author, status, and approval state; administrator may filter by status or approval state 2. Administrator selects a path pending review and inspects its pedagogical and operational information 3. For **Approve**: administrator confirms the approval; system updates the training path status to approved, makes it available within public listings, and notifies the author 4. For **Reject**: administrator clicks Reject and enters feedback explaining the reasons; system updates the status to rejected, sends the feedback to the author as a notification, and retains the path for revision
Error Scenario	1. If no training paths exist or the list cannot be retrieved, system displays an appropriate empty state or error notification 2. If the training path no longer satisfies review conditions, approval is refused 3. If the path is already approved or rejected, system displays an explanatory notice 4. If the administrator lacks permission, the action is denied 5. Persistence failures for any action retain the previous state and log the failure

• **Specification of the Use Case Curate Featured Training Paths**

Table 1.55: Use Case Specification: Curate Featured Training Paths

Title	Curate Featured Training Paths
Summary	Administrator marks approved training paths as featured or removes their featured status, and reorders featured paths to control their display sequence on the landing page
Actor	Administrator
Precondition	The administrator is authenticated with admin privileges and the target training path is approved
Post-condition	The training path featured status is updated and the landing page reflects the new selection and order
Basic Scenario	1. For **Feature / Unfeature**: administrator selects an approved training path and clicks Feature or Unfeature; system updates the featured status and confirms; the path appears in or disappears from featured listings accordingly 2. For **Reorder**: administrator navigates to the Featured Paths section; system displays all featured paths with their current display order; administrator drags and drops paths to the desired sequence; system updates order values for all featured paths and confirms
Error Scenario	1. If the training path is not approved, featuring is blocked with an explanatory error 2. If fewer than two featured paths exist, reordering displays an explanatory notice 3. Persistence failures for any action retain the current state and surface a recoverable error

• **Specification of the Use Case Manage VM Assignments**

Table 1.56: Use Case Specification: Manage VM Assignments

Title	Manage VM Assignments
Summary	Administrator views all VM template assignments across all statuses, filters to pending requests awaiting decision, and approves or rejects them with feedback to link or withhold VM templates from training units
Actor	Administrator
Precondition	The administrator is authenticated with admin privileges
Post-condition	VM assignment statuses are updated to approved or rejected; approved templates are linked to their training units and teachers are notified in either case
Basic Scenario	1. Administrator navigates to the VM Assignments section; system displays all assignments with template, training path, unit, and status; administrator may filter by status or training path 2. Administrator applies the pending filter to focus on requests awaiting decision; system displays pending assignments with teacher, training path, unit, and requested template; administrator may further filter by teacher or training path 3. Administrator selects one or more pending requests and reviews their resource requirements 4. For **Approve**: administrator confirms the approval; system validates infrastructure capacity for each request, links the VM templates to the training units, updates statuses to approved, and notifies the teachers 5. For **Reject**: administrator enters feedback explaining the reasons and confirms; system updates statuses to rejected and sends the feedback to each teacher as a notification
Error Scenario	1. If no assignments exist or the list cannot be retrieved, system displays an appropriate empty state or error notification 2. If any requested VM template no longer exists, system flags that request as invalid 3. If infrastructure capacity is insufficient for an approval, system suggests alternative templates 4. If any request is already approved or rejected, system displays an explanatory notice 5. Persistence failures for any action retain the current state and surface a recoverable error

3.2.4.6 Refinement of the Use Case Manage Reservations

Figure 1.17 presents the refinement of the Use Case Manage Reservations for the Administrator actor. This use case includes the following sub-use cases:

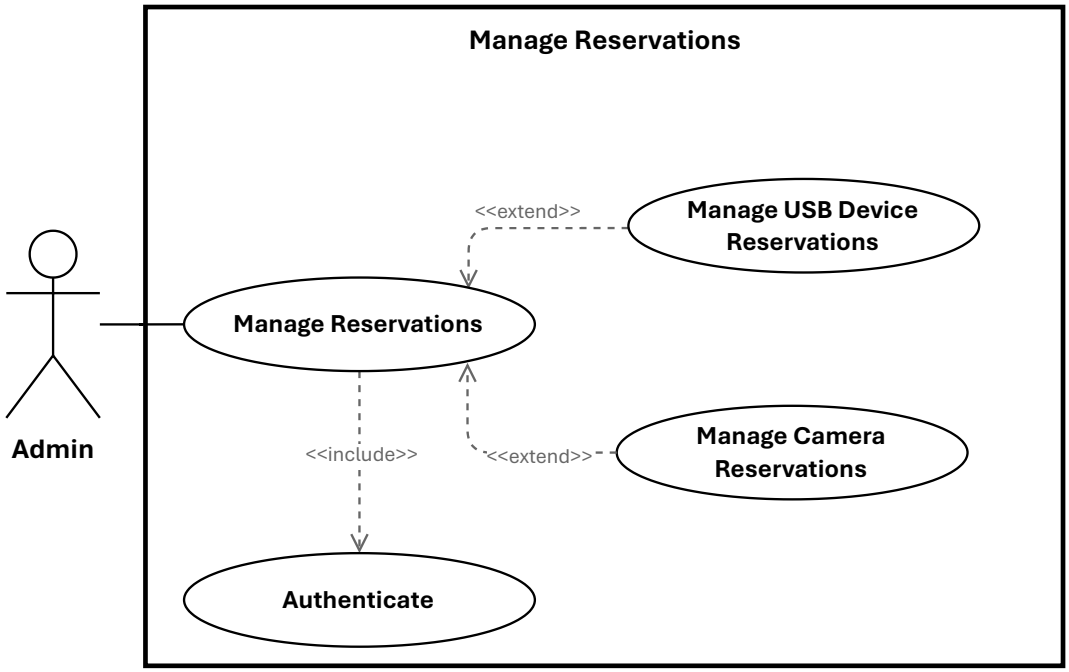

Figure 1.17: Use Case Manage Reservations

• **Specification of the Use Case Manage USB Device Reservations**

Table 1.57: Use Case Specification: Manage USB Device Reservations

Title	Manage USB Device Reservations
Summary	Administrator views all USB device reservations across all statuses, filters to pending or upcoming requests, approves or rejects pending reservations with feedback, and blocks device time slots to prevent reservations during maintenance periods
Actor	Administrator
Precondition	The administrator is authenticated with admin privileges
Post-condition	USB reservation statuses are updated to approved or rejected and engineers are notified, or a time slot is blocked and the device becomes unavailable for that period
Basic Scenario	1. Administrator navigates to the USB Reservations section; system displays all reservations with device, user, status, and time slot; administrator may filter by status or device 2. Administrator applies the **Pending** filter to focus on requests awaiting decision; system displays pending reservations with device, user, and requested time slot; administrator may filter by device or user 3. Administrator applies the **Upcoming** filter to monitor approved future reservations; system displays approved future reservations with device, user, and scheduled time slot; administrator may filter by date range or device 4. For **Approve**: administrator selects a pending request and confirms approval; system validates device availability for the requested time slot, updates the reservation status to approved, and notifies the engineer 5. For **Reject**: administrator selects a pending request, enters feedback, and confirms; system updates the status to rejected and sends the feedback to the engineer as a notification 6. For **Block Time Slot**: administrator selects a USB device, specifies a date range and reason, and confirms; system creates a blocked time slot for the device and displays the blocked periods
Error Scenario	1. If no reservations exist for the active filter, system displays an appropriate empty state 2. If any list cannot be retrieved, system shows an error notification 3. If the device is already reserved for the requested time slot, system suggests an alternative time 4. If the block time slot conflicts with existing reservations

- **Specification of the Use Case Manage Camera Reservations**

Table 1.58: Use Case Specification: Manage Camera Reservations

Title	Manage Camera Reservations
Summary	Administrator views all camera reservations, filters to pending requests, approves or rejects them with feedback, and blocks camera time slots to prevent reservations during maintenance periods
Actor	Administrator
Precondition	The administrator is authenticated with admin privileges
Post-condition	Camera reservation statuses are updated to approved or rejected and engineers are notified, or a time slot is blocked and the camera becomes unavailable for that period
Basic Scenario	1. Administrator navigates to the Camera Reservations section; system displays all reservations with camera, user, status, and time slot; administrator may filter by status or camera 2. Administrator applies the **Pending** filter; system displays pending reservations with camera, user, and requested time slot; administrator may filter by camera or user 3. For **Approve**: administrator selects a pending request and confirms approval; system validates camera availability for the requested time slot, updates the reservation status to approved, and notifies the engineer 4. For **Reject**: administrator selects a pending request, enters feedback, and confirms; system updates the status to rejected and sends the feedback to the engineer as a notification 5. For **Block Time Slot**: administrator selects a camera, specifies a date range and reason, and confirms; system creates a blocked time slot for the camera and displays the blocked periods
Error Scenario	1. If no reservations exist for the active filter, system displays an appropriate empty state 2. If any list cannot be retrieved, system shows an error notification 3. If the camera is already reserved for the requested time slot, system suggests an alternative time 4. If the block time slot conflicts with existing reservations, system displays a conflict warning 5. If the reservation is already approved or rejected, system displays an explanatory notice 6. Persistence failures for any action retain the current state

3.2.4.7 Refinement of the Use Case Manage Payout & Refunds

Figure 1.18 presents the refinement of the Use Case Manage Payout & Refunds for the Administrator actor. This use case includes the following sub-use cases:

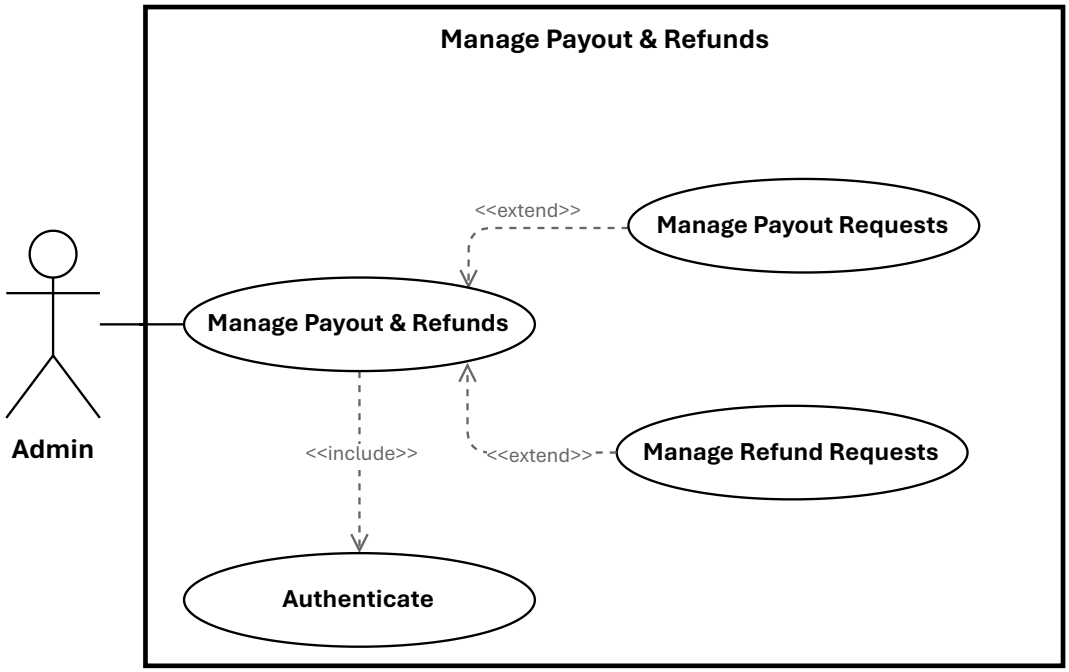

Figure 1.18: Use Case Manage Payout & Refunds

• **Specification of the Use Case Manage Payout Requests**

Table 1.59: Use Case Specification: Manage Payout Requests

Title	Manage Payout Requests
Summary	Administrator views all teacher payout requests, approves or rejects pending ones with feedback, processes approved transfers through the payment processor, and exports payout data for accounting purposes
Actor	Administrator
Precondition	The administrator is authenticated with admin privileges
Post-condition	Payout statuses are updated to approved, rejected, or processing; fund transfers are initiated for approved payouts; teachers are notified of all decisions; and payout reports are available for download
Basic Scenario	1. Administrator navigates to the Payout Requests section; system displays all requests with teacher, amount, status, and request date; administrator may filter by status or teacher 2. For **Approve**: administrator selects a pending request, reviews payout details and teacher earnings history, and confirms approval; system validates the payout amount and teacher account, updates the status to approved, and notifies the teacher 3. For **Reject**: administrator selects a pending request, enters feedback, and confirms; system updates the status to rejected and sends the feedback to the teacher as a notification 4. For **Process / Transfer**: administrator selects an approved request and confirms the transfer; system initiates the transfer through the payment processor, updates the status to processing, and logs the action for audit purposes 5. For **Export Report**: administrator applies filters (date range, status, teacher) and clicks Export; system generates and offers the report for download in CSV, PDF, or Excel format
Error Scenario	1. If no payout requests exist or the list cannot be retrieved, system displays an appropriate empty state or error notification 2. If a request is already approved or rejected, system displays an explanatory notice 3. Processing is blocked if the payout is not in approved status 4. If the transfer fails, system displays an error and marks

- **Specification of the Use Case Manage Refund Requests**

Table 1.60: Use Case Specification: Manage Refund Requests

Title	Manage Refund Requests
Summary	Administrator views pending refund requests and the full refund history, approves or rejects pending ones, and executes Stripe refunds manually when automatic processing has failed
Actor	Administrator
Precondition	The administrator is authenticated with admin privileges
Post-condition	Refund statuses are updated to approved, rejected, or completed; Stripe refunds are initiated or executed manually; engineers are notified of all decisions
Basic Scenario	1. Administrator navigates to the Refund Requests section; system displays all pending requests with engineer, amount, status, and request date; administrator may filter by status or engineer 2. Administrator navigates to the Refunds History section to review the complete transaction history; system displays all processed refunds with engineer, amount, status, and processing date; administrator may filter by status, date range, or engineer 3. For **Approve**: administrator selects a pending request, reviews the refund details and payment information, and confirms approval; system validates eligibility and amount, updates the status to approved, initiates the Stripe refund through the API, and notifies the engineer 4. For **Reject**: administrator selects a pending request, enters feedback, and confirms; system updates the status to rejected and sends the feedback to the engineer as a notification 5. For **Execute Stripe Refund** (manual fallback): administrator selects an approved request marked for manual processing and confirms execution; system calls the Stripe API, updates the refund status based on the result, and logs the action for audit purposes
Error Scenario	1. If no refund requests or history entries exist, or lists cannot be retrieved, system displays an appropriate empty state or error notification 2. If a request is already approved or rejected, system displays an explanatory notice 3. If the Stripe refund API call fails during approval, system logs the error and marks the refund for manual processing 4. If the Stripe API call fails during manual execution,

4 Sequence Diagrams

Sequence diagrams explain the chronological interaction among actors, controllers, services, models, and infrastructure components. In the current platform, the most important sequences are not limited to standard web CRUD operations; they also include payment confirmation, virtual lab provisioning, and controlled interaction with physical resources.

4.1 Training Path Enrollment and Checkout Sequence

This sequence begins when an engineer decides to access a paid or restricted training path. The user selects the training path, initiates checkout, and is redirected to the external payment flow when necessary. After Stripe confirms payment through the webhook endpoint, the platform updates the payment state and makes the related training path accessible through the learner's workspace. The process can be summarized as follows:

1. Engineer selects a training path and initiates checkout
2. Checkout controller creates the payment context
3. External payment processor handles transaction validation
4. Stripe webhook returns event data to the platform
5. System validates the signature and records the webhook event
6. Payment status is updated to completed or failed
7. Enrollment access becomes available to the engineer
8. The engineer can later consult payment history and request refund where applicable

Figure 1.19 illustrates the complete enrollment and checkout sequence, including the interaction between the frontend, the Laravel backend, and the Stripe payment processor.

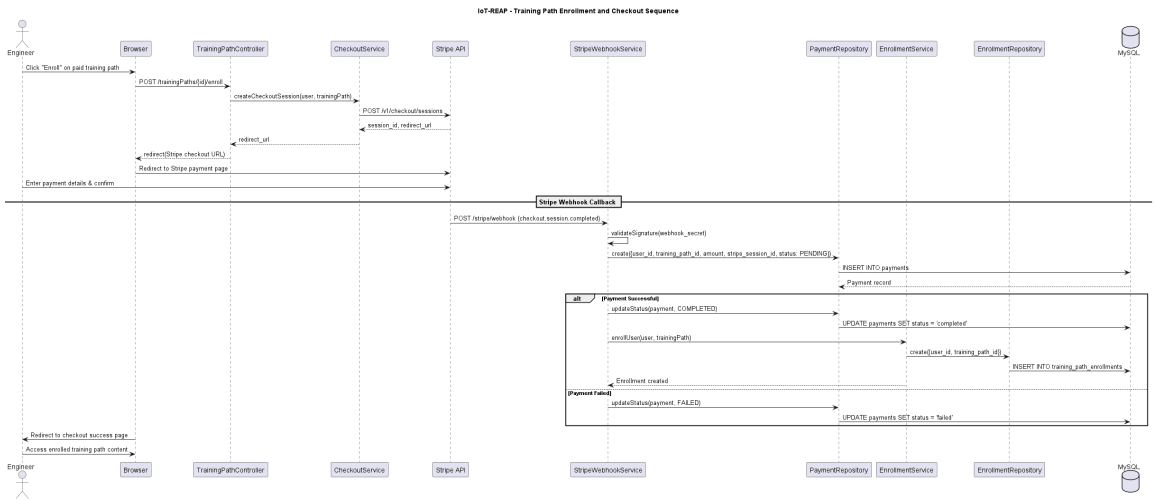

Figure 1.19: Sequence Diagram: Training Path Enrollment and Checkout

4.2 Teacher Submission for Review Sequence

This sequence models the transition from content authoring to publication governance. A teacher edits the training path and associated learning units, then submits the path for review. The platform validates ownership and readiness, changes the review status, and exposes the path to the administrator approval interface.

1. Teacher opens the teaching dashboard
2. Teacher updates the training path structure and content
3. Teacher triggers the submission action
4. Controller validates role, ownership, and submission conditions
5. Training path state changes to pending review
6. Administrator later retrieves the submission from the administrative dashboard

Figure 1.20 shows the teacher submission for review sequence, from content editing through the review status transition.

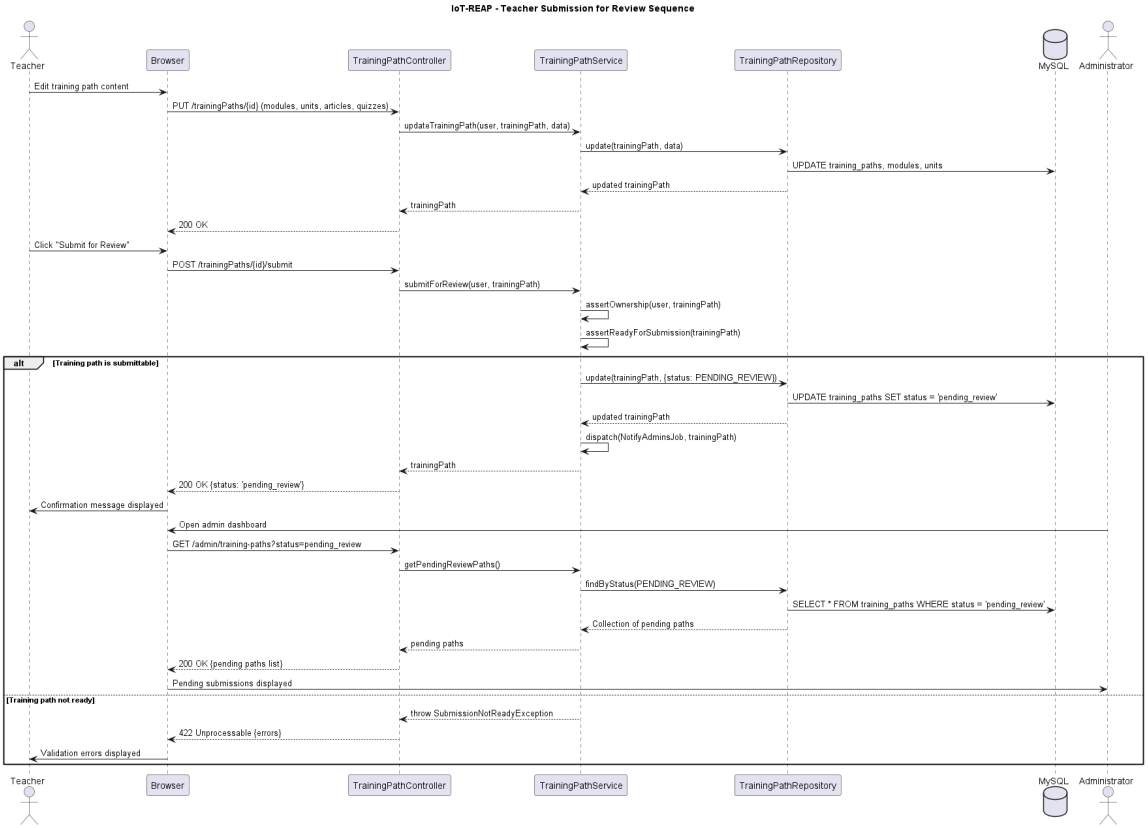

Figure 1.20: Sequence Diagram: Teacher Submission for Review

4.3 Virtual Machine Provisioning Sequence

The virtual machine provisioning sequence is a central differentiator of the platform. It links the user interface to infrastructure models and remote access capabilities.

1. Engineer sends a session creation request
2. Session controller checks authentication, verification, and provisioning permission
3. System evaluates available Proxmox servers and nodes
4. A VM session is created and associated with infrastructure metadata
5. The engineer consults the created session
6. A Guacamole token is generated on demand for remote access
7. The engineer may extend or terminate the session later

Figure 1.21 presents the virtual machine provisioning sequence, showing the interaction between the engineer, the session controller, the Proxmox API, and the Guacamole remote access gateway.

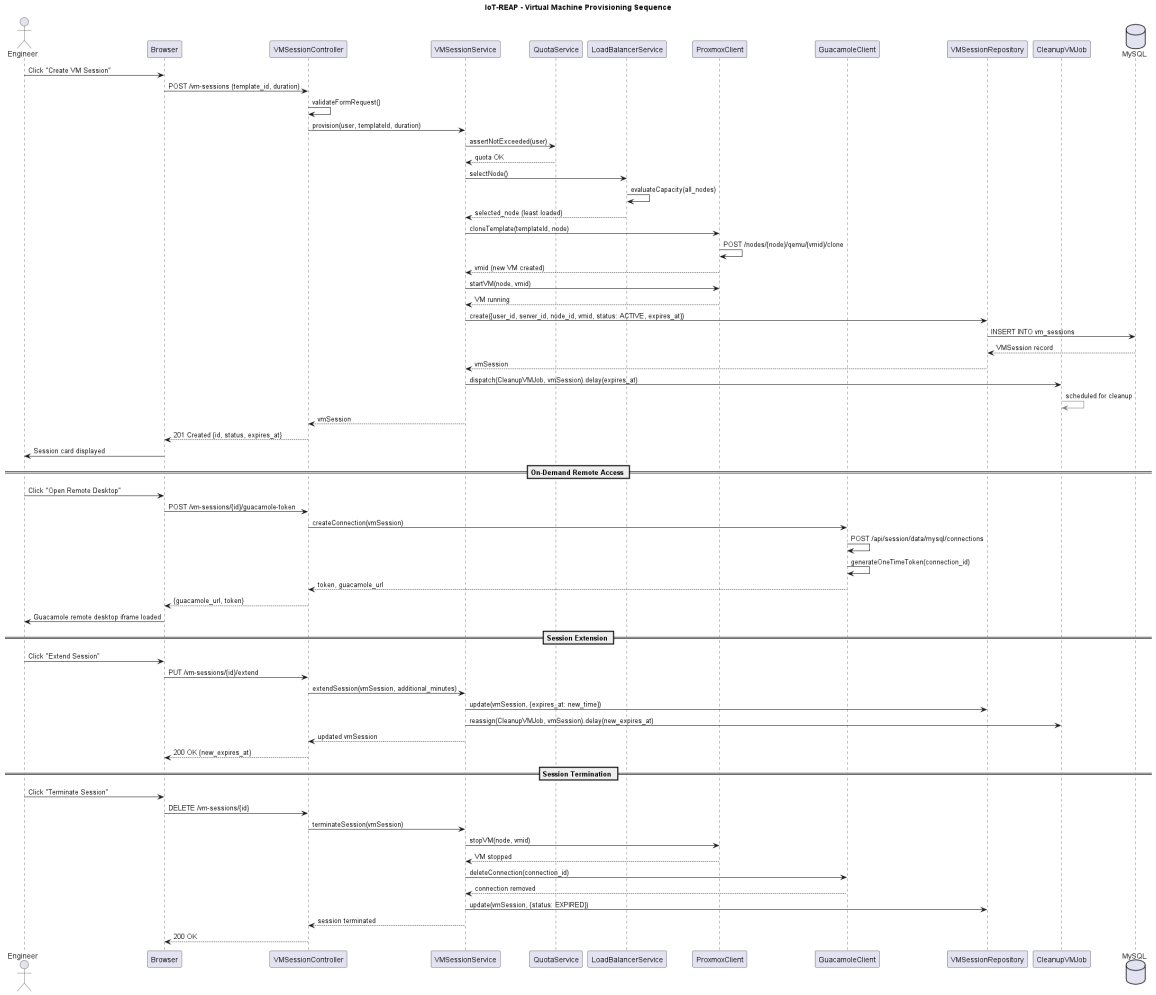

Figure 1.21: Sequence Diagram: Virtual Machine Provisioning

4.4 Hardware Attachment and Queue Sequence

This sequence governs access to USB devices attached to laboratory sessions. Because some devices are exclusive or dedicated, the workflow includes queueing behavior.

1. Engineer opens the hardware page associated with an active session
2. System returns visible devices and their current state
3. Engineer requests attachment of a selected device
4. System checks whether the device is available, attached, reserved, or dedicated
5. If the device is available, the attachment is executed
6. If the device is busy, the engineer may join a waiting queue
7. When the device becomes available, queue state is updated and notification can be issued

Figure 1.22 illustrates the hardware attachment and queue sequence, including the decision logic for device availability and the waiting queue mechanism.

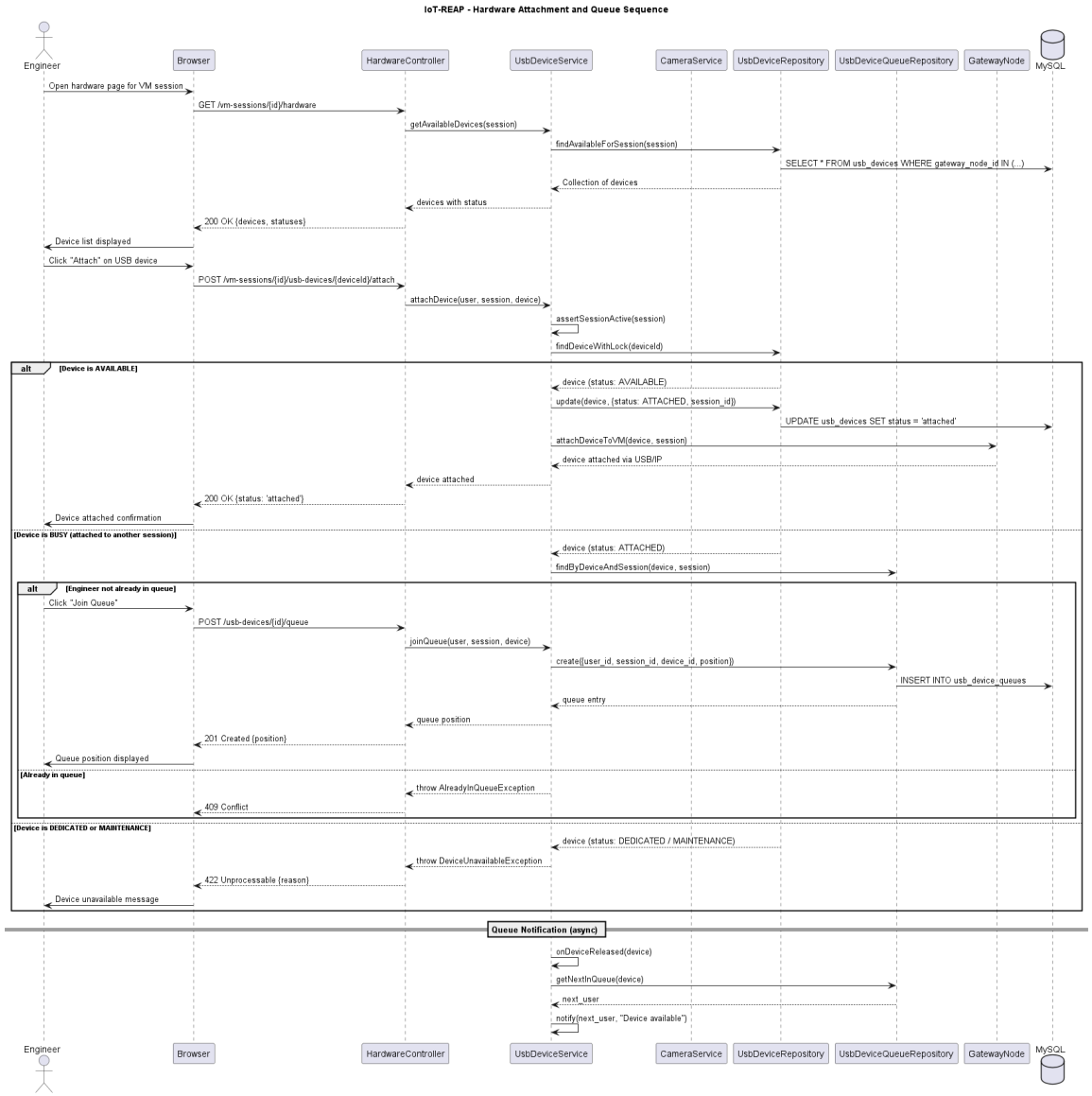

Figure 1.22: Sequence Diagram: Hardware Attachment and Queue

4.5 Quiz Attempt Submission Sequence

This sequence models the educational evaluation workflow.

1. Engineer opens the quiz associated with a training unit
2. System verifies attempt eligibility
3. A quiz attempt is created
4. Engineer submits answers
5. The platform validates and scores the attempt
6. Attempt history and completion status are updated
7. Training unit progress may be updated to reflect quiz completion or pass state

Figure 1.23 shows the quiz attempt submission sequence, from quiz opening through answer submission, scoring, and progress update.

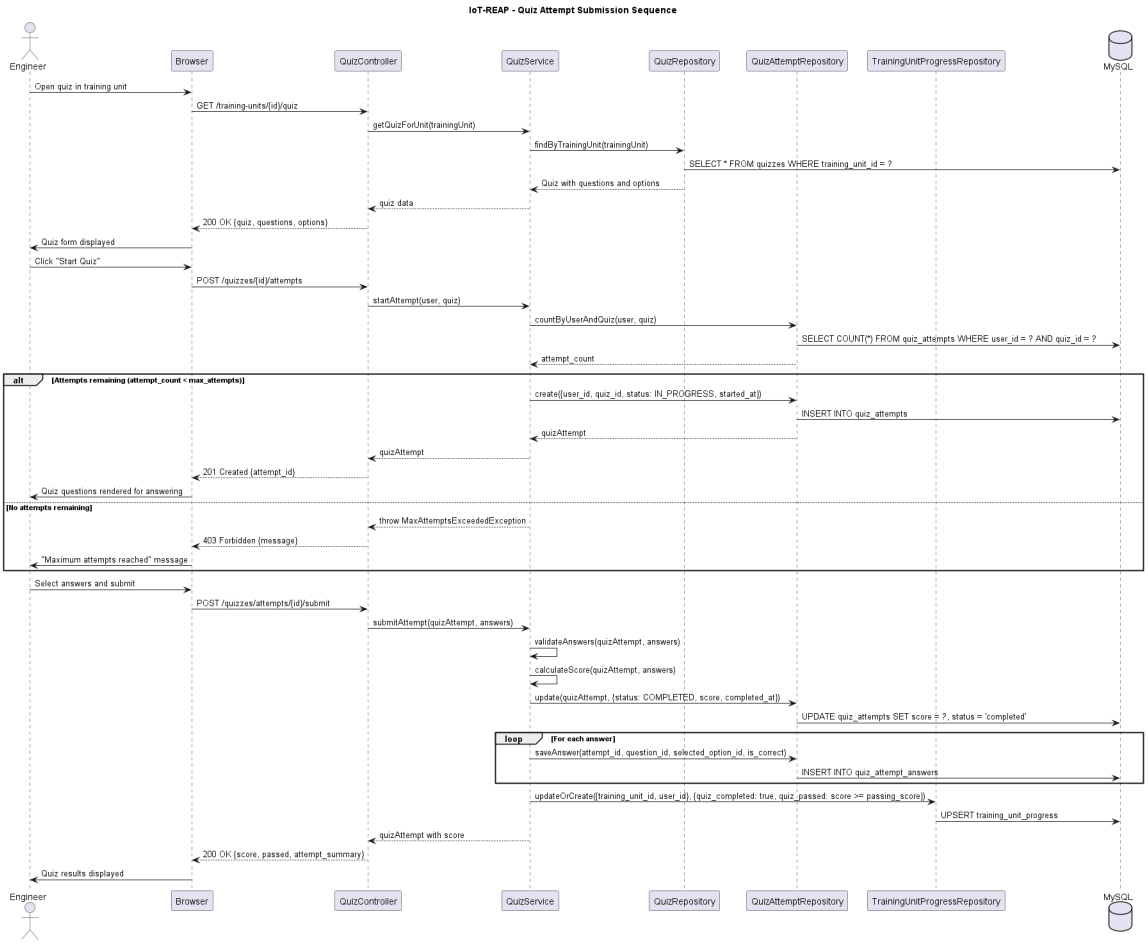

Figure 1.23: Sequence Diagram: Quiz Attempt Submission

4.6 Google OAuth Authentication Sequence

This sequence describes the social login flow when a user chooses to authenticate through Google OAuth. The process leverages Laravel Socialite to redirect the user to Google's consent screen, receive the authorization callback, and either link the Google account to an existing user or create a new account automatically. Role selection is presented when a new account is created through this pathway.

1. Guest clicks the Google login button on the authentication page
2. Laravel Socialite redirects the user to the Google OAuth consent screen
3. User authenticates with Google and grants permission
4. Google redirects back to the platform callback endpoint with an authorization code
5. Socialite exchanges the code for an access token and retrieves user profile data
6. System checks whether the Google account is already linked to an existing user
7. If linked, the user is authenticated and redirected to their dashboard
8. If not linked, a new account is created and the user is prompted to select a role
9. User selects a role and the account is finalized with the chosen role assignment

Figure 1.24 illustrates the Google OAuth authentication sequence, showing the interaction between the user, the Laravel application, and the Google identity provider.

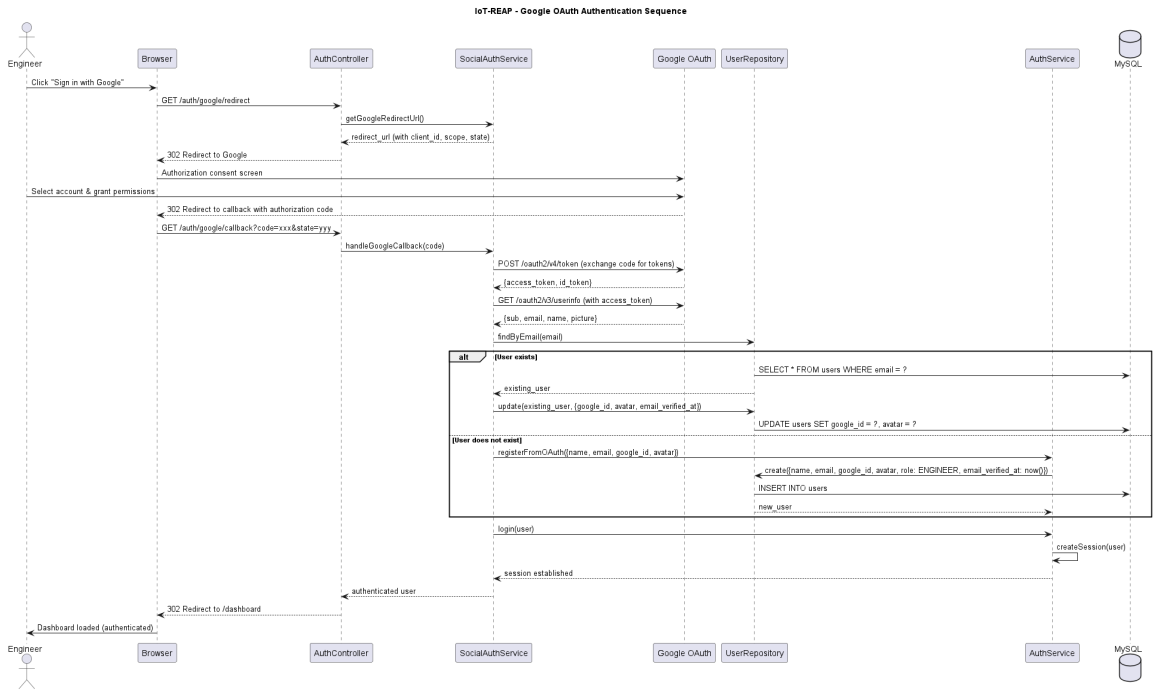

Figure 1.24: Sequence Diagram: Google OAuth Authentication

4.7 MQTT Robot Control Sequence

This sequence models the real-time command and telemetry exchange between an engineer and a robot connected through the MQTT broker. The engineer issues movement or action commands via the platform interface, which are published to the robot's command topic. The robot executes the command and publishes telemetry data back, which the platform subscribes to and relays to the engineer's dashboard.

1. Engineer opens the robot control interface within an active VM session
2. System verifies session ownership and robot access authorization
3. Engineer issues a command through the control interface
4. Platform publishes the command to the MQTT topic `robot/{robot_id}/commands` with QoS 1
5. Mosquitto broker delivers the command to the subscribed ESP32 controller
6. Robot executes the command and publishes telemetry to `robot/{robot_id}/telemetry`
7. Platform MQTT subscriber receives the telemetry update
8. Telemetry data is relayed to the engineer's interface through WebSocket or polling
9. Engineer observes the updated robot state and may issue further commands

Figure 1.25 presents the MQTT robot control sequence, showing the publish-subscribe interaction between the engineer, the Laravel backend, the Mosquitto broker, and the ESP32-controlled robot.

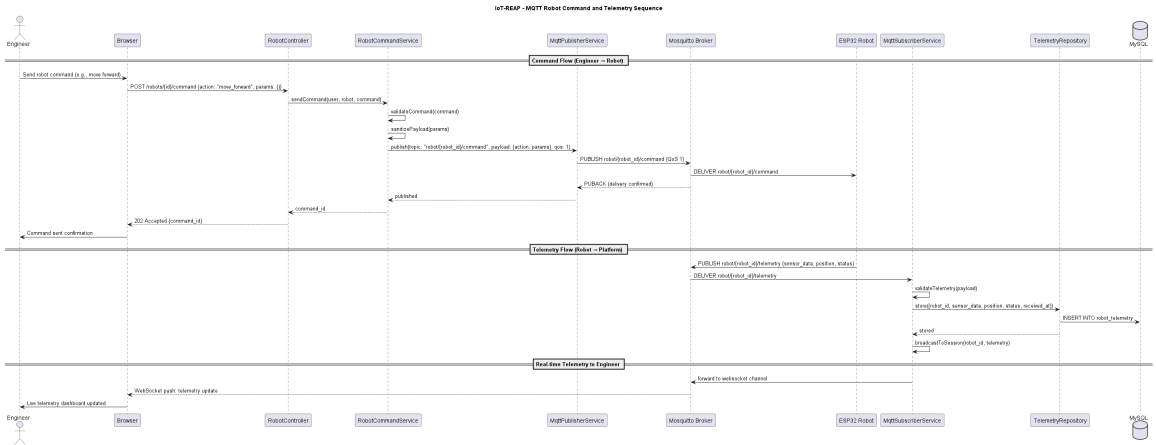

Figure 1.25: Sequence Diagram: MQTT Robot Control

4.8 Camera PTZ Control Sequence

This sequence describes how an engineer requests and exercises pan-tilt-zoom control over a PTZ-enabled camera within an active VM session. The workflow includes control request

arbitration, exclusive session control creation, movement command dispatch, and control release when the engineer finishes or the session ends.

1. Engineer opens the camera stream view within an active session
2. System displays the live camera feed and available PTZ controls
3. Engineer requests camera control
4. System checks whether the camera is currently controlled by another user
5. If available, a CameraSessionControl record is created granting exclusive control
6. Engineer issues PTZ movement commands through the interface
7. System translates each command into camera-specific movement parameters
8. Camera executes the movement and the updated frame is reflected in the stream
9. Engineer may change stream resolution during the session
10. When the engineer releases control or the session ends, the control record is removed

Figure 1.26 illustrates the camera PTZ control sequence, from control acquisition through movement commands to control release.

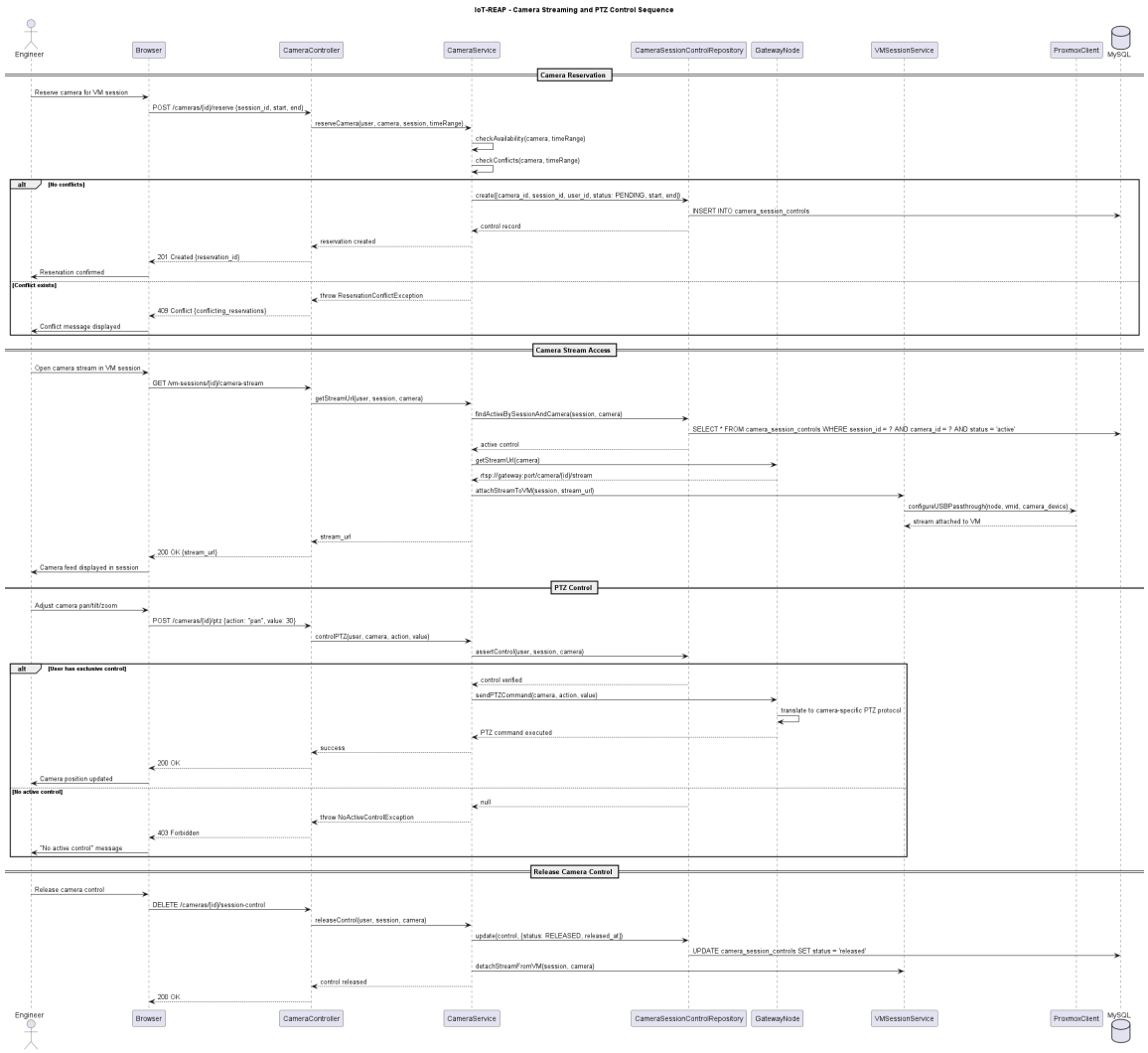

Figure 1.26: Sequence Diagram: Camera PTZ Control

4.9 Refund Processing Sequence

This sequence models the refund workflow from the engineer's initial request through administrative review and potential Stripe refund execution. It demonstrates how payment governance is handled as a multi-step auditable process rather than a simple transaction reversal.

1. Engineer submits a refund request for a completed payment
2. System validates that the payment exists and belongs to the requesting user
3. A RefundRequest record is created with pending status
4. Administrator opens the refund review interface
5. Administrator reviews the payment context and refund reason
6. Administrator approves or rejects the refund request
7. If approved, the system initiates a refund through the Stripe API
8. Stripe processes the refund and returns the result
9. System updates the refund request status and the payment state accordingly
10. Engineer is notified of the refund decision and outcome

Figure 1.27 presents the refund processing sequence, showing the interaction between the engineer, the administrator, the Laravel backend, and the Stripe refund API.

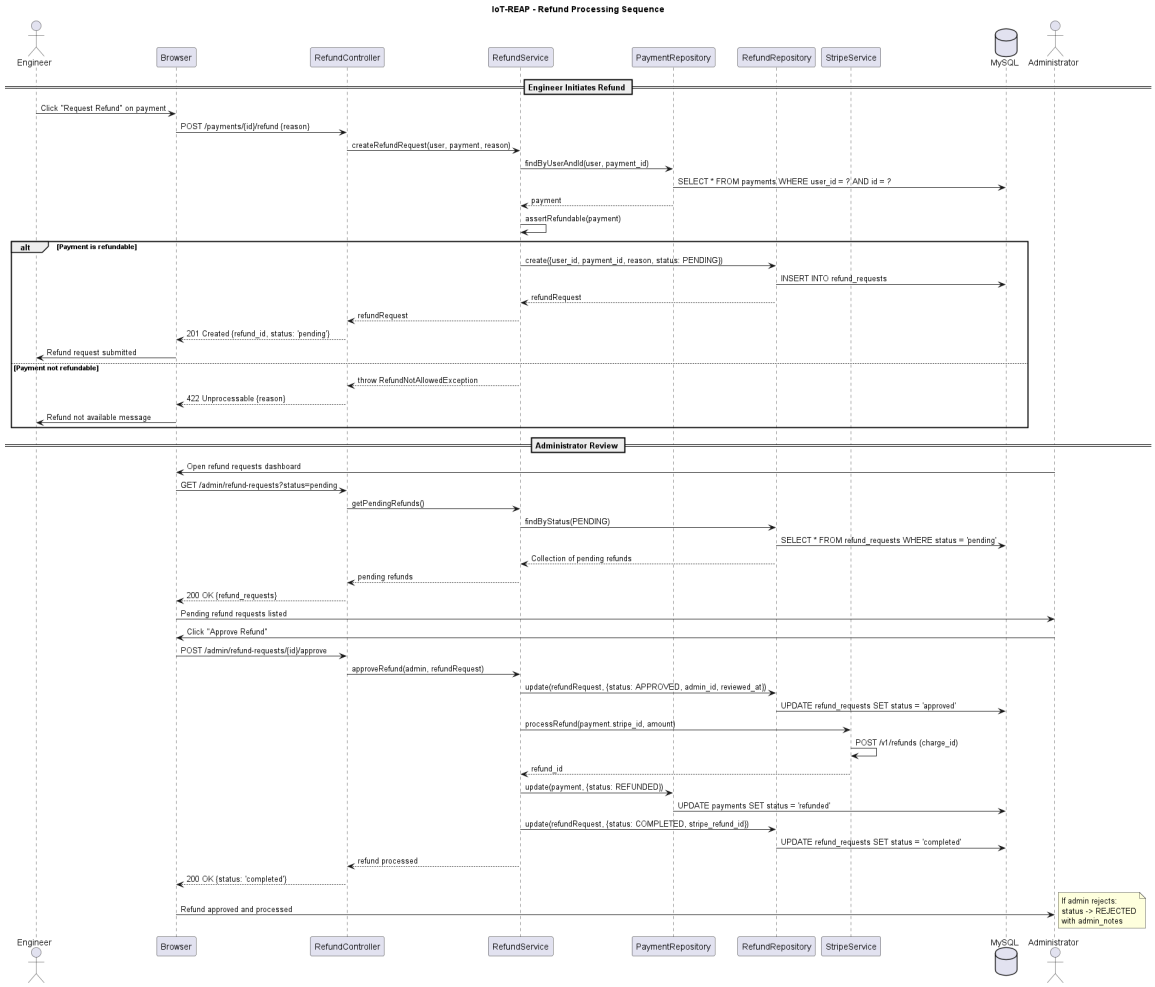

Figure 1.27: Sequence Diagram: Refund Processing

5 Class Diagram

The class diagram of the current platform is broader than that of a conventional learning portal because it combines academic content, learner engagement, payment processing, virtual machine infrastructure, device reservation, and operational monitoring. The conceptual model can therefore be organized into interrelated domains.

5.1 User and Access Domain

User Class: The `User` class is the central identity entity of the platform. It stores authentication and profile information, enforces role-aware behavior through helper methods such as teacher and administrator checks, and links a person to enrollments, taught training paths, quiz attempts, payments, and notifications. It is the pivot between educational participation and infrastructure usage.

Notification and Activity Classes: The `Notification`, `ActivityLog`, and `SystemAlert` classes support communication, auditability, and operational supervision. Notifications deliver user-level updates, activity logs record significant actions, and system alerts capture infrastructure or platform incidents that require administrative attention.

5.2 Learning Content Domain

TrainingPath Class: The `TrainingPath` class is the main pedagogical aggregate. It belongs to an instructor, contains modules and training units, receives enrollments, collects reviews and payments, and exposes status-dependent scopes such as approved or pending review. It represents the complete educational offering published on the platform.

TrainingPathModule and TrainingUnit Classes: `TrainingPathModule` structures a training path into ordered thematic groups, while `TrainingUnit` represents the granular learning unit. A training unit can contain one article, one video, one quiz, one virtual machine assignment, and multiple forms of progress tracking. These classes define the educational hierarchy used by both teachers and learners.

Article, Video, and Caption Classes: `Article` stores textual educational content and derived metrics such as word count and estimated reading time. `Video` stores media-related metadata, processing status, and streaming access paths. `Caption` complements the video model with subtitle and accessibility support. Together, these classes model the multimedia core of the learning experience.

Quiz Domain Classes: The assessment subsystem is formed by `Quiz`, `QuizQuestion`, `QuizQuestionOption`, `QuizAttempt`, and `QuizAttemptAnswer`. These classes represent the authored evaluation structure, learner answers, scoring context, attempt limits, and result history. They support both teacher-side authoring and learner-side evaluation.

Progress and Participation Classes: `TrainingUnitProgress`, `VideoProgress`, `TrainingUnitNote`, `TrainingPathEnrollment`, `TrainingPathReview`, `DiscussionThread`, `ThreadReply`, and `ThreadVote` model learner participation. They collectively express completion state, note taking, enrollment membership, reputation, peer interaction, and social learning dynamics.

Certificate Class: The `Certificate` class materializes completion outcomes. It belongs to a user and a training path, and provides verification and download URLs that support public authenticity checks.

5.3 Commerce and Revenue Domain

Payment and RefundRequest Classes: The `Payment` class records financial transactions associated with users and training paths. It stores amount-related data, completion state, and refund relationships. `RefundRequest` adds the post-payment governance layer by representing approval, rejection, processing, and failure states for refund handling.

PayoutRequest Class: The `PayoutRequest` class represents teacher-side revenue withdrawal requests. It links the requesting user with administrative review and processing decisions, thereby connecting platform earnings to instructor compensation workflows.

5.4 Virtual Laboratory and Infrastructure Domain

VMSession Class: The `VMSession` class represents an engineer's temporary remote laboratory instance. It belongs to a user, server, and compute node, and connects to attached devices and hardware queues. It is the main bridge between educational access and infrastructure provisioning.

ProxmoxServer and ProxmoxNode Classes: `ProxmoxServer` models the virtualization cluster endpoint, while `ProxmoxNode` models the compute node hosted under that server. These classes represent capacity, status, and allocation context required for provisioning VM sessions.

TrainingUnitVMAssignment Class: The `TrainingUnitVMAssignment` class links pedagogical content to technical infrastructure by allowing a training unit to request and receive an approved virtual machine environment. It connects teacher requests to administrator approval and provides a controlled way to embed hands-on lab work inside a training path.

GatewayNode, UsbDevice, Camera, and Reservation Classes: `GatewayNode` models the hardware gateway responsible for exposing physical resources. `UsbDevice` represents a manageable peripheral, while `Camera` represents a visual sensor that may also be associated with a robot or USB device. Reservation behavior is represented through `Reservation`, `UsbDeviceReservation`, and `CameraReservation`. These classes support availability planning, conflicts, approval, activation, and maintenance-aware control.

CameraSessionControl and Queueing Classes: `CameraSessionControl` represents temporary exclusive control of a camera inside a VM session. `UsbDeviceQueue` models waiting behavior when multiple users contend for limited hardware resources. These classes express concurrency control within the remote laboratory domain. Figure 1.28 presents the complete class diagram of the IoT-REAP platform, organized into the domain packages described above: User and Access, Learning Content, Commerce and Revenue, and Virtual Laboratory and Infrastructure.

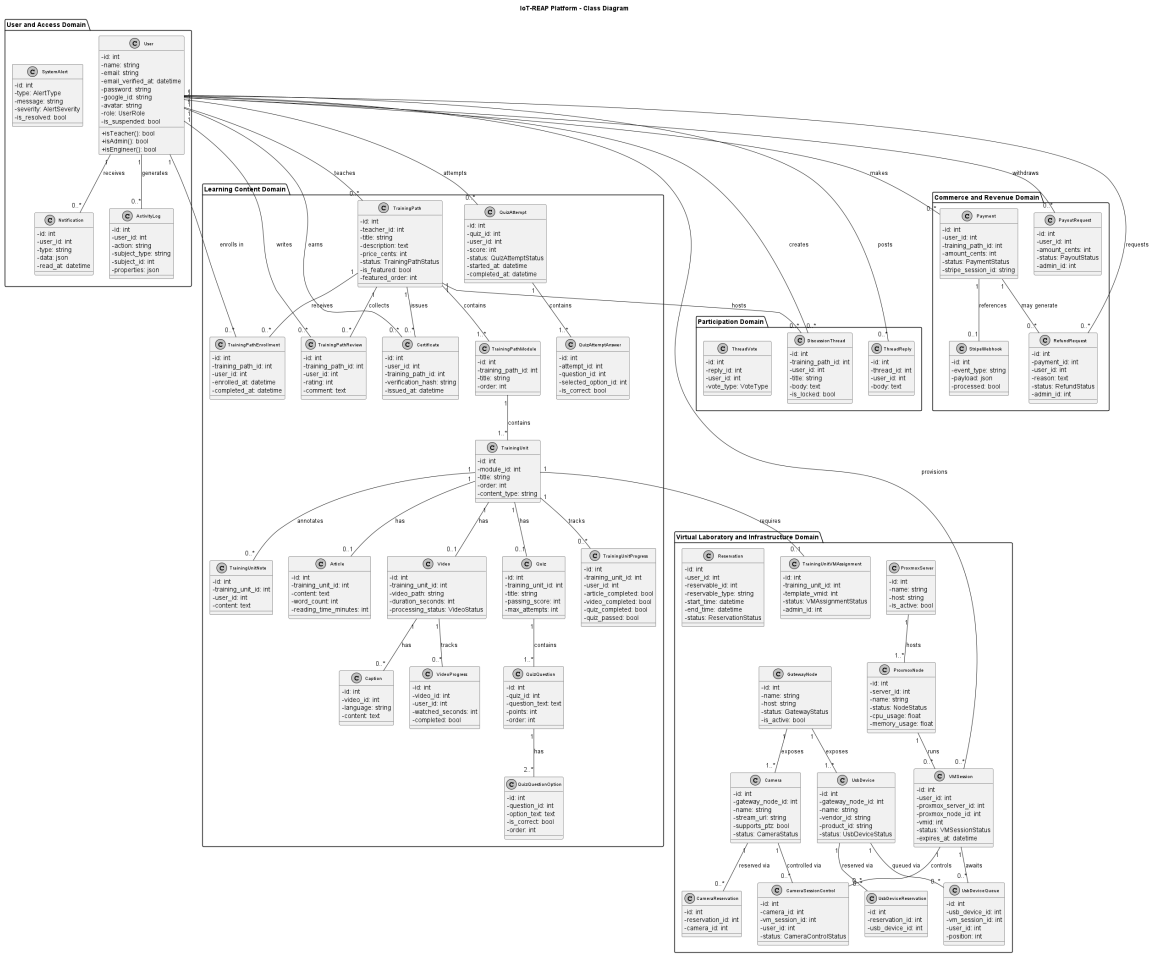

Figure 1.28: Class Diagram of the IoT-REAP Platform

6 Physical Data Model

The physical data model follows the domain structure exposed by the Eloquent models and route organization. At database level, the platform is centered on the `users` table, which is referenced by multiple transactional and educational tables. From this anchor, the model expands into five major relational clusters.

6.1 Learning and Authoring Tables

The training subsystem is organized around `training_paths`, `training_path_modules`, and `training_units`. The relationship is hierarchical: one training path contains many modules, and one module contains many training units. Additional one-to-one or one-to-many tables extend training units with content and pedagogy, including `articles`, `videos`, `captions`, `quizzes`, `quiz_questions`, `quiz_question_options`, `quiz_attempts`, and `quiz_attempt_answers`.

Progress-oriented tables such as `training_unit_progress`, `video_progress`, and `training_unit_notes` preserve learner-state evolution over time.

6.2 Participation and Certification Tables

Collaborative learning is represented by `discussion_threads`, `thread_replies`, and `thread_votes`. Satisfaction and reputation are represented by `training_path_reviews`. Enrollment membership is represented by `training_path_enrollments`. Completion evidence is represented by `certificates`, which reference both user and training path and expose verifiable hashes for public consultation.

6.3 Payment and Revenue Tables

Commercial activity is represented by `payments`, `refund_requests`, `payout_requests`, and `stripe_webhooks`. These tables preserve financial state transitions, external payment event traceability, and instructor revenue workflows. The data model ensures that one payment belongs to one user and one training path, while a payment may receive multiple related refund records depending on business logic.

6.4 Virtual Machine and Infrastructure Tables

Remote laboratory persistence is built around `vm_sessions`, `proxmox_servers`, `proxmox_nodes`, `training_unit_vm_assignments`, and user-specific connection preference tables such as `guacamole_connection_preferences` and `user_vm_connection_default_profiles`. This cluster stores compute allocation, session duration, infrastructure ownership, and preferred access settings.

6.5 Hardware and Reservation Tables

The IoT and device-control cluster includes `gateway_nodes`, `usb_devices`, `cameras`, `camera_session_controls`, `reservations`, `usb_device_reservations`, `camera_reservations`, and `usb_device_queues`. These tables model the availability, attachment, queueing, maintenance, and scheduling logic required to safely expose real laboratory resources to remote users.

6.6 Governance and Monitoring Tables

Operational governance is supported by `notifications`, `activity_logs`, `system_alerts`, `node_credentials_logs`, and derived or statistical tables such as `daily_training_path_stats`. Together, these structures provide auditability, accountability, administrative observability, and monitoring support for a platform that mixes education with infrastructure control.

Overall, the physical data model reflects a normalized but highly interconnected design. Foreign keys enforce ownership and navigation between users, content, payments, reservations, and infrastructure entities. This organization makes the database suitable for both transactional consistency and operational extensibility. Figure 1.29 presents the entity-relationship diagram of the IoT-REAP physical data model, showing the major tables, their attributes, and the foreign key relationships across the six relational clusters.

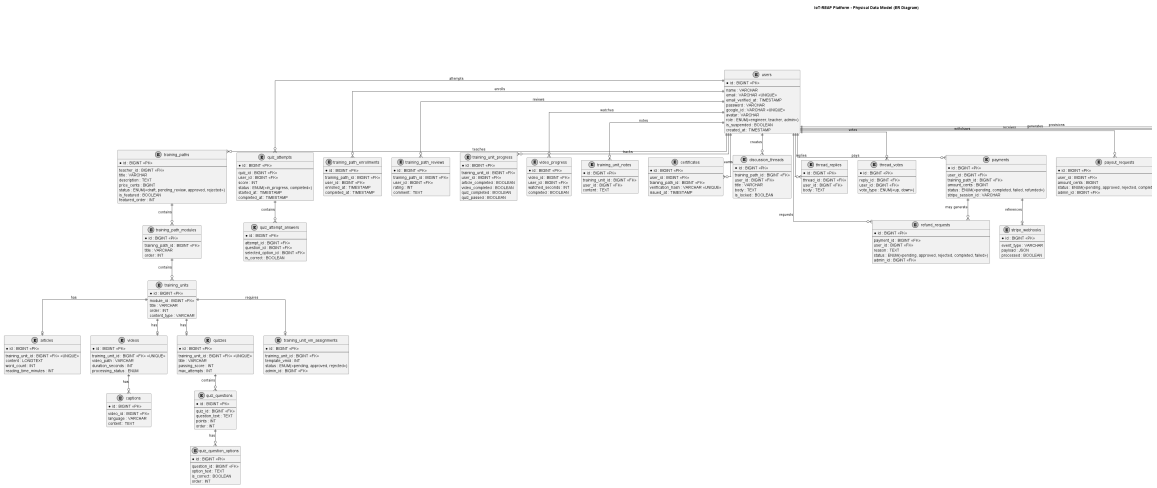

Figure 1.29: Entity-Relationship Diagram of the IoT-REAP Physical Data Model

7 Component Architecture Diagram

The component architecture of IoT-REAP reflects the separation of concerns between the frontend presentation layer, the backend application layer, and the external infrastructure services. The platform is built as a monolithic Laravel application with an Inertia.js-bridged React frontend, communicating with Proxmox VE for virtualization, Apache Guacamole for remote desktop proxying, an MQTT broker for robot telemetry and control, Stripe for payment processing, and USB/IP gateways for hardware device access.

The frontend layer consists of React components organized into pages, shared components, custom hooks, Zustand state stores, and typed API modules. The backend layer follows the request lifecycle pattern: FormRequest validation, thin Controllers, Service classes containing

business logic, Repository classes for data access, Eloquent Models for domain representation, and API Resources for response shaping. Background processing is handled through Laravel Queues with jobs for VM provisioning, video processing, and cleanup operations.

The infrastructure layer includes Proxmox VE clusters for compute virtualization, Apache Guacamole for protocol-adaptive remote access (RDP, VNC, SSH), Mosquitto MQTT broker for real-time robot communication, Stripe for payment and refund processing, and USB/IP gateway nodes for physical device exposure. Laravel Echo and Pusher provide real-time event broadcasting for notifications and live updates.

Figure 1.30 presents the component architecture diagram, showing the major subsystems and their communication pathways.

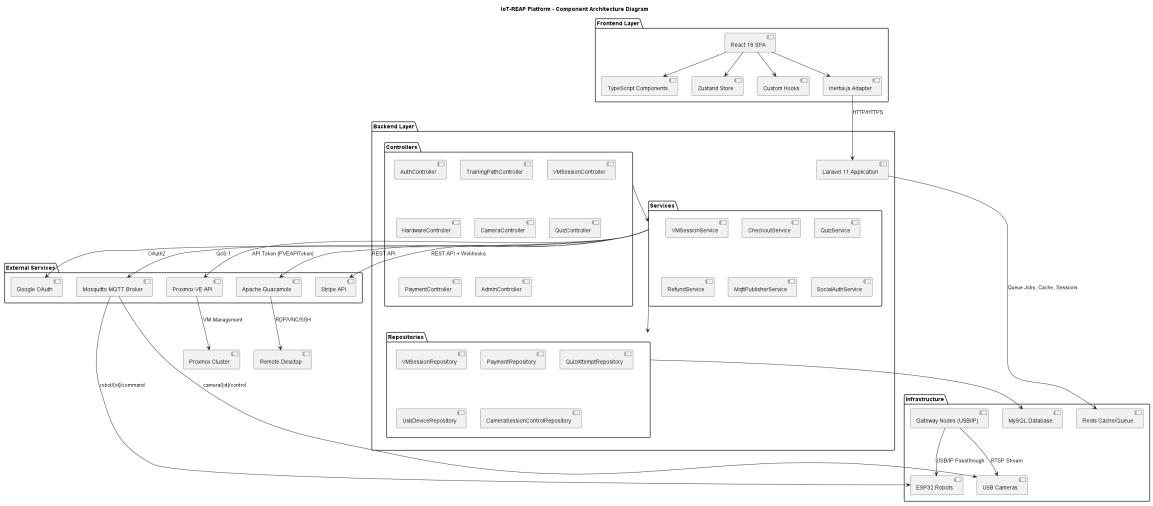

Figure 1.30: Component Architecture Diagram of the IoT-REAP Platform

8 Deployment Diagram

The deployment of IoT-REAP spans multiple physical and virtual nodes, reflecting the hybrid nature of an educational platform that integrates cloud-hosted application services with on-premises laboratory infrastructure. The deployment architecture consists of the following principal nodes.

The **Web Application Server** hosts the Laravel application, the React frontend served through Inertia.js, the Nginx reverse proxy, and the PHP-FPM process manager. This node also runs the MySQL database, the Redis cache and session store, and the Laravel queue worker processes. The Mosquitto MQTT broker may be co-located on this server or deployed on a separate node depending on traffic requirements.

The **Proxmox VE Cluster** comprises one or more physical servers running the Proxmox hypervisor. Each cluster node hosts virtual machines that are provisioned on demand for

engineer lab sessions. The cluster is accessed through the Proxmox API using token-based authentication.

The **Apache Guacamole Gateway** runs as a separate service that proxies remote desktop protocols (RDP, VNC, SSH) to the engineer's browser. It connects to the provisioned VMs through the internal network and exposes a WebSocket-based client interface.

The **USB/IP Gateway Nodes** are physical machines running the USB/IP kernel module and custom gateway management software. Each gateway node exposes attached USB devices and cameras to the virtual machines through IP-based USB forwarding. The gateways register with the main application and report device availability and health status.

The **Stripe Payment Service** operates as an external SaaS endpoint, receiving checkout sessions and webhook callbacks over HTTPS. It is not deployed within the platform infrastructure but is integrated through the Stripe SDK and webhook signature verification.

Figure 1.31 presents the deployment diagram, showing the physical nodes, the software artifacts deployed on each node, and the communication protocols between them.

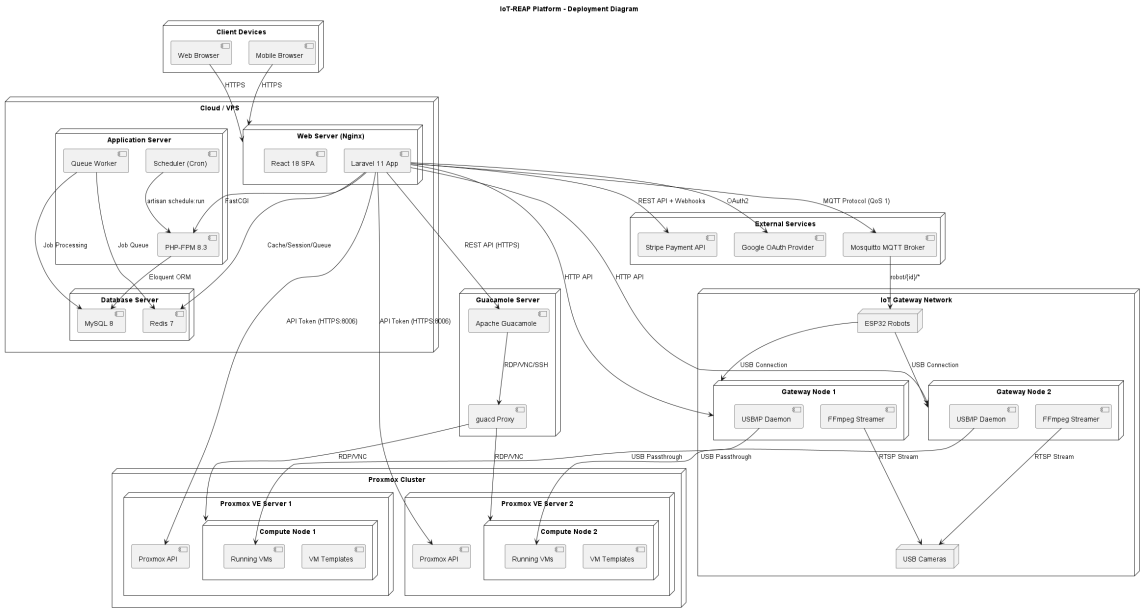

Figure 1.31: Deployment Diagram of the IoT-REAP Platform

Conclusion

This chapter presented the conceptual analysis of the current IoT REAP platform. The rewrite replaced the outdated assessment-oriented description with a model that corresponds to the implemented system: a role-based learning and remote laboratory platform integrating public discovery, course authoring, learner progression, payment workflows, virtual machine provisioning, device and camera reservations, and administrative governance. The actor

analysis, requirements specification, use case refinements, sequence and activity descriptions, class model, physical data model, component architecture, and deployment diagram together provide a coherent conceptual foundation for the technical realization discussed in the following chapters.